

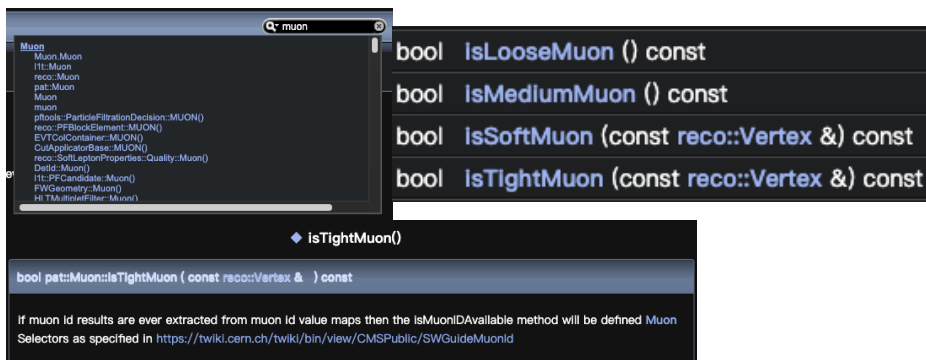
[Download the article](#)

[Video tutorial](#)

0-order

Tips before viewing

This note will only introduce the logical relationship of the program and the variable settings that need to be used, and will add the author's simple understanding of the physical meaning. There will be many details that are not covered, such as why this variable is used in this way, why this function needs to add this variable, etc. I don't know how to explain some of these problems, and most of them are related to the underlying code. This cannot be explained clearly in this short note. You need to find the usage of the corresponding variables in [manual](#) on the website. For example, you can find the MuonID function in `pat::Muon`, and you will find that `isSoftMuon` requires a `reco::Vertex` variable, but `isLooseMuon` does not. You can also find a more detailed introduction to MuonID there. You can also find more details about CMSSW work in [WorkBookCMSSWFramework < CMSPublic < TWiki \(cern.ch\)](#). So this note can only be your starting point. If you want to become a true master of high-energy physics analysis, you must be a physicist driven by interest.

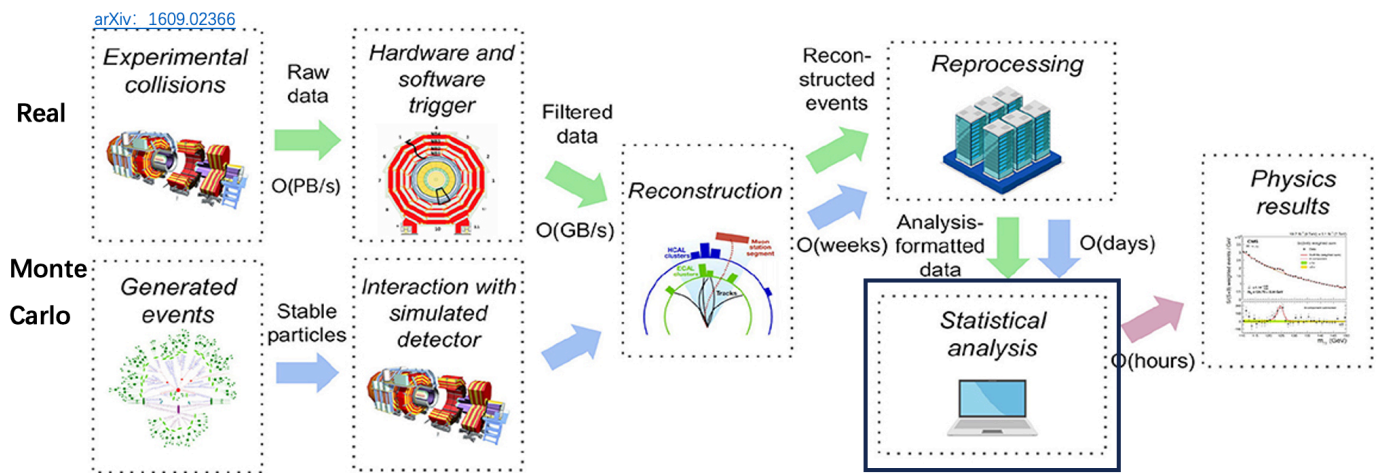


CMS data

Before entering into CMS physical analysis, you need to have a general understanding of how CMS data comes from. Usually we have two sources of data, one is real data (Real Data) obtained by collecting particle information generated by LHC collisions in the real world, and the other is simulated data (Monte Carlo Data) obtained by using theoretical models combined with detector simulations. Both types of data have undergone similar processing and are finally stored in CERN's data center ([CERN Data Centre](#)) to facilitate offline physical analysis by high-energy physicists. These processes include GENSIM->RAW->RECO->SKIM

Processing LHC Data

- **GENSIM (Generation & Simulation)** refers to particle impact cases directly detected by the detector (particle impact events simulated with Monte Carlo)
- **RAW** In fact, most of the impact cases are boring, and the impact frequency is much higher than the limit that the computer can handle. These cases will be preliminarily screened in real time through the hardware level of the detector itself or the software program, and only the cases that we are interested in will be left.
- **RECO (Reconstruction)** The particles produced by the collision will leave electrical signals in the detector. This step will convert these signals into particle spatial positions and physical quantities with physical meaning for analysis. The meaning of "reconstruction" is that we can turn back the axis of time and deduce what the particle looked like before.
- **SKIM** This step is exactly what this note introduces, which is the offline filtering corresponding to real-time filtering like Trigger.



■ GENSIM -> RAW -> RECO -> SKIM

We are here!

PS: The data after RECO will actually be processed once to generate data in the root format named **AOD (Analysis Object Data)** and stored. Later, because there is too much information stored, AOD is slimmed down and there are data types of **mini-AOD** and **nano-AOD**. There will be some differences between them, which you need to understand in specific analysis.

the term

- **Grid Certificate**

CMS data is distributed in computing centers around the world. To access this data, you need to pass security authentication. The user needs to load the personal certificate and generate a temporary proxy:

```
1 | voms-proxy-init --voms cms
```

In this way, grid resources can be used (such as submitting tasks, reading data, etc.).

- **CMSSW (CMS Software)**

CMS's official software framework for data processing, simulation and physical analysis. All analysis codes usually run in the CMSSW environment and need to be established and entered into the workspace through commands such as `cmsrel`, `cmsenv`.

- **Condor Jobs**

HTCondor is a commonly used task scheduling system that can distribute a large number of tasks to computing nodes for running. Users write submission scripts, and Condor is responsible for task queuing, distribution, and status management.

- **CRAB Jobs**

CRAB (CMS Remote Analysis Builder) is a high-level tool provided by CMS, encapsulated based on Condor and grid systems. It simplifies the process for users to run analysis tasks on distributed computing resources, and only needs to provide configuration files to automatically submit, monitor and manage tasks.

Preparation

If you are not ready to jump in, maybe this manual can take you to experience the joy of data analysis [Getting Started with CMS MiniAOD Open Data](#). If you want to play the full version, this [CMS Workbook](#) should take you from entry to proficiency.

Here are the empty files in following path, you can copy it to your work directory using `cmsrel CMSSW_13_0_13` cms-software version because of the root file we will use. I also dropped some hints in the corresponding folder named as '_hint'. So, if and only if you are stuck, you can get some helps with it. Never forget `cmsenv` before your work.

ps: The hepthu cluster will have problems during cmsRun. It is recommended to perform the following work on the lxplus cluster.

```
/home/storage0/users/zhufeng/formymultilep_learning/UserCode/
/afs/cern.ch/user/z/zhuf/public/UserCode/
/nucms/home/zhufeng/formutilep_learning/UserCode
```

1-File structure

```
1 | CMSSW_13_0_13
```

```

2  /src
3  └─UserCode
4      └─MuMuEEPat
5          └─BuildFile.xml
6          └─doc
7          └─interface
8              └─MuMuEEPat.h
9                  └─MuMuEEPat.h_hint
10         └─mumueepat_cfg.py
11         └─python
12             └─mumueepat_cfi.py
13                 └─__pycache__
14                     └─mumueepat_cfi.cpython-39.pyc
15         └─src
16             └─MuMuEEPat.cc
17                 └─MuMuEEPat.cc_hint
18         └─test
19             └─f8e93985-e14c-4a8a-b28b-f8cceb3c878e.root
20             └─myntuple.C_hint
21             └─myntuple.C.rootmap
22             └─myntuple.h
23             └─Runjobs.C
24             └─runMultiLepPAT_dataRun3_miniAOD.py

```

Before starting work, please check whether your file structure is correct against the above file tree, especially the structure of `CMSSW_13_0_13/src/UserCode/MuMuEEPat`. The `_hint` file is the complete code as a prompt, please use it appropriately.

When you modify this program, you need to compile it. After the compilation is passed, you can enter the test folder and use some files to test your code. You need to use the following command to compile your code in the upper-level directory of UserCode, that is, under the path `CMSSW_13_0_13/src`, and you must ensure that there can only be one `.cc` and **KEEP_4 under** `UserCode/MuMuEEPat/src` and `UserCode/MuMuEEPat/interface` **Files with** suffix (referring to files with duplicate content, of course you can have multiple files with `.cc` and `.h` suffixes with different contents), so please do not use `.cc` and `.h` as suffixes when backing up your own code.

```

1  scramv1 b ProjectRename
2  scramv1 b clean #When you enter a new working directory you need the above two lines to clear some caches
3  scramv1 b      #When you modify your code, use this sentence to compile it.

```

The `.cc` and `.h` files are the main analysis programs. Here is a brief explanation of the structure of the program. For the input data file, that is, the AOD root format file mentioned before, it contains the physical information in multiple collision cases (Events), and there must be no physical relationship between the cases. Therefore, we only need to write the screening process for one case (Events), and just keep looping for other cases. Such loops have been preset under this framework, which is the analyze part in the figure below. Other parts are prepared for storing data.

```

src
├── KinematicParticleVertexFitter.cc_old
├── MultiLepPAT.cc_copy
├── MuMuEEPat.cc
└── MuMuEEPat.cc_7_27ok
    
```

→ The main procedure of the analysis

```

1 MuMuEEPat::MuMuEEPat(const edm::ParameterSet& iConfig)
2 {}
3
4 MuMuEEPat::~MuMuEEPat()
5 {}
6
7 void MuMuEEPat::beginJob() {}
8
9 void MuMuEEPat::beginRun(edm::Run const& iRun, edm::EventSetup const& iSetup) {}
10
11 void MuMuEEPat::analyze(const edm::Event& iEvent, const edm::EventSetup& iSetup) { }
12
13 void
14 MuMuEEPat::endJob() {}
15
16 DEFINE_FWK_MODULE(MuMuEEPat);
17 #endif
    
```

```

graph TD
    iConfig[iConfig] --> beginJob[beginJob]
    beginJob --> analyze1[analyze]
    analyze1 --> analyze2[analyze]
    analyze2 --> endJob[endJob]
    
```

iConfig: 初始化你在.h文件中定义的变量，需要与你定义的顺序一致

beginJob: 定义你的Tree和Branch，为数据存储做准备

analyze (LOOP iEvent): 会对所有的事例进行重复，在每个事例结束后会将数据存储在Tree中，并清除为下个事例做准备

endJob: 存储你的Tree，并清空缓存

The MuMuEEPat.h header file is used to declare function variables, etc., and does not contain specific implementation details.

```

1 class MuMuEEPat : public edm::one::EDAnalyzer<edm::one::SharedResources>{
2
3 public://Public members can be accessed from outside the class, and any object or function can access the
4 public members of the class.
5     explicit MuMuEEPat(const edm::ParameterSet&); //initialization
6     ~MuMuEEPat();
7
8 private://Private members can only be accessed from within the class. Only member functions of a class can
9 access private members of the class
10     virtual void beginJob();
11     virtual void beginRun(edm::Run const& iRun, edm::EventSetup const& iSetup);
12     virtual void analyze(const edm::Event&, const edm::EventSetup&);
13     virtual void endJob();
14     //You don't need to understand what it's doing, copy it directly!
15 };
16 #endif
    
```

The specific analysis process is in the MuMuEEPat.cc source file.

```

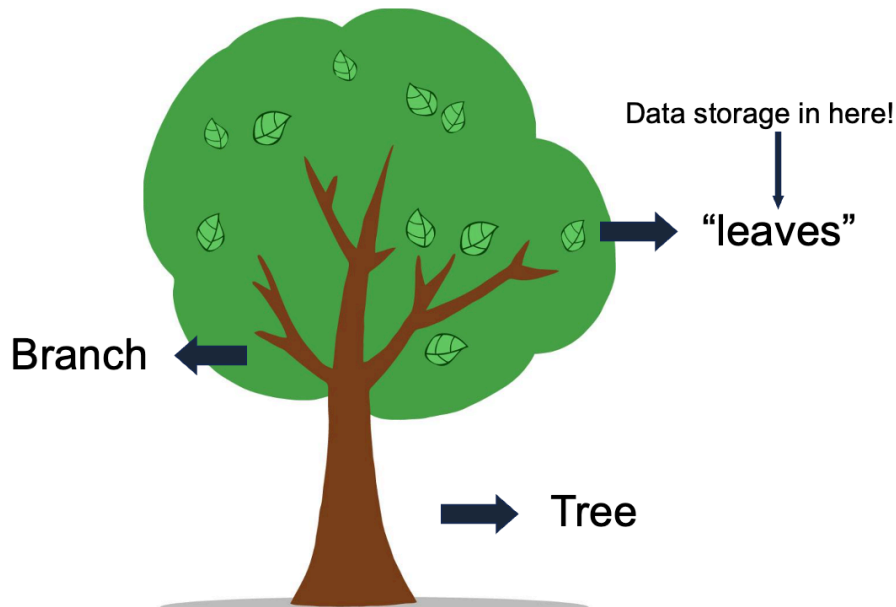
1 MuMuEEPat::MuMuEEPat(const edm::ParameterSet& iConfig)
2 {
3     //Initialize variables
4 }
5
6 MuMuEEPat::~MuMuEEPat()
7 {
8     //Clear cache?
9 }
10
11 void MuMuEEPat::beginJob() {
12     //In the part that runs before the analyze loop, you need to define the variables you need here.
13 }
14
15 void MuMuEEPat::beginRun(edm::Run const& iRun, edm::EventSetup const& iSetup) {
16 }
17
18
19 void MuMuEEPat::analyze(const edm::Event& iEvent, const edm::EventSetup& iSetup) {
20     //The main analysis program will automatically cycle
    
```

```

21 }
22
23 void
24 MuMuEEPat::endJob() {
25 //During the loop of analyze, data will be continuously filled in the tree you defined. After the loop ends,
26 //you need to write the tree to a file and perform necessary cache cleaning.
27 }
28 DEFINE_FWK_MODULE(MuMuEEPat);

```

2-Create and use tree



We use the Tree!

We need to put many branches on a tree and store the data in the leaves corresponding to the branches. As said before, the best time to plant a tree is before (beginjob). Correspondingly, the vectors in tree and branch must be defined in the .h file. After analyzing, the tree is already lush, and we need to output the tree in endjobs. As for how to stuff leaves on this tree in analyze, read on.

```

1 //MuMuEEPat.h
2 ...
3 //define token begin
4 edm::EDGetTokenT<edm::View<pat::Muon> > gtpatmuonToken_;
5 edm::ESGetToken<MagneticField, IdealMagneticFieldRecord> magneticFieldToken_;
6 //define token end
7
8 TTree* X_One_Tree_;//Tree is necessary. For convenience, here we only use one Tree to store all variables.
9 vector<double> *mumuonlyMass, *mumuonlyMassErr;//Define your tree and variables in the .h file. Here we use
10 //vector arrays to store data.
11 ...

```

```

1 //MuMuEEPat.cc
2 MuMuEEPat::MuMuEEPat(const edm::ParameterSet& iConfig)
3 :magneticFieldToken_(esConsumes<MagneticField, IdealMagneticFieldRecord>()),
4 ...,
5 mumuonlyMass(0),mumuonlyMassErr(0),
6 ...//This is where all the variables you defined in .h initialize parameters. The initialization needs to be
7 //consistent with the order of definition. Note that there is no comma in the last variable.
8 {
9 //extract token begin

```

```

9  gtpatmuonToken_ = consumes<edm::View<pat::Muon> >(edm::InputTag("slimmedMuons"));
10 //extract token end
11 }
12 void MuMuEFPat::beginJob() {
13     edm::Service < TFileService > fs;
14     X_One_Tree_ = fs->make < TTree > ("X_data", "X_Data");//Define the name of the tree, which is equivalent to
    planting a tree and naming it X_Data
15     X_One_Tree_->Branch("mumuonlyMass",&mumuonlyMass);
16     X_One_Tree_->Branch("mumuonlyMassErr",&mumuonlyMassErr);//Define the branch with the same name and point to
    the corresponding vector. This means that the tree you planted before has grown branches.
17 }
18 void MuMuEFPat::analyze(const edm::Event& iEvent, const edm::EventSetup& iSetup)//It is worth noting that
    this part of the program will only filter one collision case and does not write a statement to loop through
    all cases. Don't worry, it will automatically loop through all cases when running. This is why the program is
    divided into different parts.
19 {
20     ...
21     for(...){//Loop through all muons in a case. You can set conditions to filter the muons you want and obtain
        the physical quantities of these muons.
22         ...
23         mumuonlyMass->push_back(...);
24         mumuonlyMassErr->push_back(...);//The main way to fill in data into an array
25         ...
26     }
27     ...
28
29     //clear
30     X_One_Tree_->Fill();//The leaves have been obtained in the previous for loop. With this sentence, you can put
        these data on the corresponding branches and fill your tree.
31     mumuonlyMass->clear();mumuonlyMassErr->clear();//Because you need to perform the analyze part of the muon in
        the next case, you need to clear your array to avoid data duplication.
32 }
33 void MuMuEFPat::endJob() {
34     X_One_Tree_->Write();//After running the analyze part, use this command to save your Tree
35     delete X_One_Tree_;//Delete and clean cache
36 }

```

summary: If you want to add a new variable, you need to:

1. Define your variable names in the .h file, paying attention to the order and data type
2. Initialize your variables in iConfig in the .cc file, pay attention to the order
3. In beginJob in the .cc file, define the branch corresponding to the Tree where you want to store the variable, usually with the same name as the variable.
4. Fill in your data into variables in the analyze in the .cc file, and perform necessary clearing in the final clear section.



In the next two sections, we will try to output the physical information of the muon pair in the example. The first step is to build a for loop to pair the muon in the example. The second step is to fit the muon track to obtain the physical information and output it.

3-for loop start

Before we can understand the logic of a program, we need to physically understand what we are about to do. For the root file we want to analyze, it contains all the collision information generated during a certain period of detector operation, and we call a collision an event (Event), so we can say that a root file contains many Events. Each collision may produce many particles, but most of these particles cannot be directly observed by us. Instead, the final particles produced by their decay are detected by us through detectors, and then the dynamic information of the parent particles before decay can be deduced through dynamic calculations. For example, we now want to find a process like $J/\psi \rightarrow \mu^+ \mu^-$, where the J/ψ particle cannot be directly observed by the detector, and its decay time limits its flight time, and it is too late to hit the detector. Therefore, we will study the final particle μ particles that it decays out of, and detect the generation of J/ψ particles through the dynamic information of μ particles.

We can first look at one of the collision cases. For example, in the second collision case, we detected a total of 8 μ particles. These 8 μ particles left position information on the detector, which is called a track. Through the track and magnetic field information, we can obtain their dynamic information. Because what we need to study is a pair of μ particles, now we need to combine 8 μ particles in pairs. Logically speaking, there will be 28 possible combinations. However, we consider that the J/ψ particles are electrically neutral, which requires that the charges of the two μ particles add up to 0. Such conditions (and other conditions that need to be met) will result in the number of final pairing combinations being less than 28.

In addition to charge screening, another important condition is vertex fitting. Because in terms of physics, one particle decays into two particles, and the trajectories of the two decayed particles must come out from one point. The same is definitely true for the detector, so it is necessary to require that the two μ particles that are assumed to be decayed by J/ψ also come out from the same point. We usually call it a vertex (vertex), but limited by the accuracy of the detector, we will use vertex probability to measure the possibility that they come from the same vertex.

Events	N_muon	parameters (raw muon)			
1	4	Muon index(i)	$p_{x\mu i}$	$p_{y\mu i}$	$p_{z\mu i}$
2	8	1	$p_{x\mu 1}$	$p_{y\mu 1}$	$p_{z\mu 1}$
3	3	2	$p_{x\mu 2}$	$p_{y\mu 2}$	$p_{z\mu 2}$
4	12	3	$p_{x\mu 3}$	$p_{y\mu 3}$	$p_{z\mu 3}$
...	...	...	...	...	...
n	m	8	$p_{x\mu 8}$	$p_{y\mu 8}$	$p_{z\mu 8}$

Possible components $C_8^2 = 28$

Muon pair		parameters					
Index(n)	comp	J/ψ (muon pair)		Fit muon 1		Fit muon 2	
		$p_{xJ/\psi n}$	...	$p'_{x\mu n}$	...	$p''_{x\mu n}$	...
1	[1,3]	$p_{xJ/\psi 1}$	...	$p'_{x\mu 1}$	...	$p''_{x\mu 1}$	...
2	[2,6]	$p_{xJ/\psi 2}$	...	$p'_{x\mu 2}$	...	$p''_{x\mu 2}$	...
3	[2,8]	$p_{xJ/\psi 3}$	...	$p'_{x\mu 3}$	...	$p''_{x\mu 3}$	...
...	...	...	...	...	...	...	...
14	[5,8]	$p_{xJ/\psi 14}$	...	$p'_{x\mu 14}$	...	$p''_{x\mu 14}$	...

$p'_{x\mu 2} = p'_{x\mu 3} ?$
 $p_{x\mu 1} = p'_{x\mu 1} ?$

After understanding the above physical relationship, the next step is to implement it in the program.

The following parts omit the definition, initialization and cleaning process of some variables. Please refer to the previous chapters for corresponding operations.

```

1 //MuMuEEP.h
2 private:
3 //define token begin
4 edm::EDGetTokenT<edm::View<pat::Muon> > gtpatmuonToken_;//Get the token of pat::Muon
5 edm::ESGetToken<MagneticField, IdealMagneticFieldRecord> magneticFieldToken_;//Get the token of MagneticField
6 //define token end

```

```

1 //MuMuEEP.cc
2 MuMuEEP::MuMuEEP(const edm::ParameterSet& iConfig)
3 :magneticFieldToken_(esConsumes<MagneticField, IdealMagneticFieldRecord>())
4 {
5 //extract token
6 gtpatmuonToken_ = consumes<edm::View<pat::Muon> >(edm::InputTag("slimmedMuons"));//The token of pat::Muon
  imported into the .h file can be extracted here.
7 //extract token
8 }

```

The gtpatmuonToken_ here is a member variable, which saves a Token. This Token indicates that the module requires an object of type pat::Muon, and the data source is a data set named "slimmedMuons" (specified by edm::InputTag). This dataset name is defined in the data processing chain and is used to identify the data source.

You can use the following method to check what this tag is (usually Displaced is not used, as for other rootfiles, it is not clear what the situation is)

```

1 $ edmDumpEventContent f8e93985-e14c-4a8a-b28b-f8cceb3c878e.root
2 ...
3 vector<pat::Muon> "slimmedDisplacedMuons" " " "PAT"
4 vector<pat::Muon> "slimmedMuons" " " "PAT"
5 ...

```

pat::Muon is an object type that represents muons, which is a type of "Physics Object" used in analysis. PAT (Physics Analysis Toolkit) is a package in CMSSW that provides a set of standard high-level physical object types, such as electrons, muons, jets, etc. The pat::Muon object contains various information about the muon, such as momentum, energy, trajectory, etc.

```

1 //MuMuEFPat.cc
2 void MuMuEFPat::analyze(const edm::Event& iEvent, const edm::EventSetup& iSetup)
3 {
4   edm::Handle<edm::View<pat::Muon>> thePATMuonHandle; //Use edm::Handle to ensure that these data objects are
5   valid and correctly initialized when accessing data. Define a smart pointer thePATMuonHandle
6   iEvent.getByToken(gtppatmuonToken_, thePATMuonHandle); //Extract the previous token and store it in
7   thePATMuonHandle
8   edm::View<pat::Muon>::const_iterator iMuon1; //Define two constant iterators pointing to the pat::Muon object
9   in edm::View<pat::Muon>, which are read-only and cannot be modified.
10  edm::View<pat::Muon>::const_iterator iMuon2;
11  for (iMuon1 = thePATMuonHandle->begin(); iMuon1 != thePATMuonHandle->end(); ++iMuon1) {
12    TrackRef muTrack1 = iMuon1->track(); //The difference from reco::Track below is that muTrack1 can be used
13    for lightweight operations. For example, in order to see whether the track is valid, the one below directly
14    reconstructs the value for storage of track data. This is done to save the amount of calculations. In short,
15    muTrack1 is more suitable for some operations.
16    if (muTrack1.isNull()) {continue;} //Check whether the tracks of the two muons are valid. If the track is
17    invalid, skip the following steps.
18    reco::Track recoMu1 = *iMuon1->track(); //The effective muon track is reconstructed and stored in recoMu1
19    for (iMuon2 = iMuon1 + 1; iMuon2 != thePATMuonHandle->end(); ++iMuon2) { //In order to find two muons
20      paired together, you need to loop n(n-1)/2 times
21      TrackRef muTrack2 = iMuon2->track();
22      if (muTrack2.isNull()) {continue;}
23      reco::Track recoMu2 = *iMuon2->track();
24      if ( (iMuon1->charge()+iMuon2->charge()) == 0 ) { //Need mu+mu-charge sum equal to 0
25        //next part
26      }
27    }
28  }
29 }
30 }
31 }
32 }
33 }
34 }
35 }
36 }
37 }
38 }
39 }
40 }
41 }
42 }
43 }
44 }
45 }
46 }
47 }
48 }
49 }
50 }
51 }
52 }
53 }
54 }
55 }
56 }
57 }
58 }
59 }
60 }
61 }
62 }
63 }
64 }
65 }
66 }
67 }
68 }
69 }
70 }
71 }
72 }
73 }
74 }
75 }
76 }
77 }
78 }
79 }
80 }
81 }
82 }
83 }
84 }
85 }
86 }
87 }
88 }
89 }
90 }
91 }
92 }
93 }
94 }
95 }
96 }
97 }
98 }
99 }
100 }
101 }
102 }
103 }
104 }
105 }
106 }
107 }
108 }
109 }
110 }
111 }
112 }
113 }
114 }
115 }
116 }
117 }
118 }
119 }
120 }
121 }
122 }
123 }
124 }
125 }
126 }
127 }
128 }
129 }
130 }
131 }
132 }
133 }
134 }
135 }
136 }
137 }
138 }
139 }
140 }
141 }
142 }
143 }
144 }
145 }
146 }
147 }
148 }
149 }
150 }
151 }
152 }
153 }
154 }
155 }
156 }
157 }
158 }
159 }
160 }
161 }
162 }
163 }
164 }
165 }
166 }
167 }
168 }
169 }
170 }
171 }
172 }
173 }
174 }
175 }
176 }
177 }
178 }
179 }
180 }
181 }
182 }
183 }
184 }
185 }
186 }
187 }
188 }
189 }
190 }
191 }
192 }
193 }
194 }
195 }
196 }
197 }
198 }
199 }
200 }
201 }
202 }
203 }
204 }
205 }
206 }
207 }
208 }
209 }
210 }
211 }
212 }
213 }
214 }
215 }
216 }
217 }
218 }
219 }
220 }
221 }
222 }
223 }
224 }
225 }
226 }
227 }
228 }
229 }
230 }
231 }
232 }
233 }
234 }
235 }
236 }
237 }
238 }
239 }
240 }
241 }
242 }
243 }
244 }
245 }
246 }
247 }
248 }
249 }
250 }
251 }
252 }
253 }
254 }
255 }
256 }
257 }
258 }
259 }
260 }
261 }
262 }
263 }
264 }
265 }
266 }
267 }
268 }
269 }
270 }
271 }
272 }
273 }
274 }
275 }
276 }
277 }
278 }
279 }
280 }
281 }
282 }
283 }
284 }
285 }
286 }
287 }
288 }
289 }
290 }
291 }
292 }
293 }
294 }
295 }
296 }
297 }
298 }
299 }
300 }
301 }
302 }
303 }
304 }
305 }
306 }
307 }
308 }
309 }
310 }
311 }
312 }
313 }
314 }
315 }
316 }
317 }
318 }
319 }
320 }
321 }
322 }
323 }
324 }
325 }
326 }
327 }
328 }
329 }
330 }
331 }
332 }
333 }
334 }
335 }
336 }
337 }
338 }
339 }
340 }
341 }
342 }
343 }
344 }
345 }
346 }
347 }
348 }
349 }
350 }
351 }
352 }
353 }
354 }
355 }
356 }
357 }
358 }
359 }
360 }
361 }
362 }
363 }
364 }
365 }
366 }
367 }
368 }
369 }
370 }
371 }
372 }
373 }
374 }
375 }
376 }
377 }
378 }
379 }
380 }
381 }
382 }
383 }
384 }
385 }
386 }
387 }
388 }
389 }
390 }
391 }
392 }
393 }
394 }
395 }
396 }
397 }
398 }
399 }
400 }
401 }
402 }
403 }
404 }
405 }
406 }
407 }
408 }
409 }
410 }
411 }
412 }
413 }
414 }
415 }
416 }
417 }
418 }
419 }
420 }
421 }
422 }
423 }
424 }
425 }
426 }
427 }
428 }
429 }
430 }
431 }
432 }
433 }
434 }
435 }
436 }
437 }
438 }
439 }
440 }
441 }
442 }
443 }
444 }
445 }
446 }
447 }
448 }
449 }
450 }
451 }
452 }
453 }
454 }
455 }
456 }
457 }
458 }
459 }
460 }
461 }
462 }
463 }
464 }
465 }
466 }
467 }
468 }
469 }
470 }
471 }
472 }
473 }
474 }
475 }
476 }
477 }
478 }
479 }
480 }
481 }
482 }
483 }
484 }
485 }
486 }
487 }
488 }
489 }
490 }
491 }
492 }
493 }
494 }
495 }
496 }
497 }
498 }
499 }
500 }
501 }
502 }
503 }
504 }
505 }
506 }
507 }
508 }
509 }
510 }
511 }
512 }
513 }
514 }
515 }
516 }
517 }
518 }
519 }
520 }
521 }
522 }
523 }
524 }
525 }
526 }
527 }
528 }
529 }
530 }
531 }
532 }
533 }
534 }
535 }
536 }
537 }
538 }
539 }
540 }
541 }
542 }
543 }
544 }
545 }
546 }
547 }
548 }
549 }
550 }
551 }
552 }
553 }
554 }
555 }
556 }
557 }
558 }
559 }
560 }
561 }
562 }
563 }
564 }
565 }
566 }
567 }
568 }
569 }
570 }
571 }
572 }
573 }
574 }
575 }
576 }
577 }
578 }
579 }
580 }
581 }
582 }
583 }
584 }
585 }
586 }
587 }
588 }
589 }
590 }
591 }
592 }
593 }
594 }
595 }
596 }
597 }
598 }
599 }
600 }
601 }
602 }
603 }
604 }
605 }
606 }
607 }
608 }
609 }
610 }
611 }
612 }
613 }
614 }
615 }
616 }
617 }
618 }
619 }
620 }
621 }
622 }
623 }
624 }
625 }
626 }
627 }
628 }
629 }
630 }
631 }
632 }
633 }
634 }
635 }
636 }
637 }
638 }
639 }
640 }
641 }
642 }
643 }
644 }
645 }
646 }
647 }
648 }
649 }
650 }
651 }
652 }
653 }
654 }
655 }
656 }
657 }
658 }
659 }
660 }
661 }
662 }
663 }
664 }
665 }
666 }
667 }
668 }
669 }
670 }
671 }
672 }
673 }
674 }
675 }
676 }
677 }
678 }
679 }
680 }
681 }
682 }
683 }
684 }
685 }
686 }
687 }
688 }
689 }
690 }
691 }
692 }
693 }
694 }
695 }
696 }
697 }
698 }
699 }
700 }
701 }
702 }
703 }
704 }
705 }
706 }
707 }
708 }
709 }
710 }
711 }
712 }
713 }
714 }
715 }
716 }
717 }
718 }
719 }
720 }
721 }
722 }
723 }
724 }
725 }
726 }
727 }
728 }
729 }
730 }
731 }
732 }
733 }
734 }
735 }
736 }
737 }
738 }
739 }
740 }
741 }
742 }
743 }
744 }
745 }
746 }
747 }
748 }
749 }
750 }
751 }
752 }
753 }
754 }
755 }
756 }
757 }
758 }
759 }
760 }
761 }
762 }
763 }
764 }
765 }
766 }
767 }
768 }
769 }
770 }
771 }
772 }
773 }
774 }
775 }
776 }
777 }
778 }
779 }
780 }
781 }
782 }
783 }
784 }
785 }
786 }
787 }
788 }
789 }
790 }
791 }
792 }
793 }
794 }
795 }
796 }
797 }
798 }
799 }
800 }
801 }
802 }
803 }
804 }
805 }
806 }
807 }
808 }
809 }
810 }
811 }
812 }
813 }
814 }
815 }
816 }
817 }
818 }
819 }
820 }
821 }
822 }
823 }
824 }
825 }
826 }
827 }
828 }
829 }
830 }
831 }
832 }
833 }
834 }
835 }
836 }
837 }
838 }
839 }
840 }
841 }
842 }
843 }
844 }
845 }
846 }
847 }
848 }
849 }
850 }
851 }
852 }
853 }
854 }
855 }
856 }
857 }
858 }
859 }
860 }
861 }
862 }
863 }
864 }
865 }
866 }
867 }
868 }
869 }
870 }
871 }
872 }
873 }
874 }
875 }
876 }
877 }
878 }
879 }
880 }
881 }
882 }
883 }
884 }
885 }
886 }
887 }
888 }
889 }
890 }
891 }
892 }
893 }
894 }
895 }
896 }
897 }
898 }
899 }
900 }
901 }
902 }
903 }
904 }
905 }
906 }
907 }
908 }
909 }
910 }
911 }
912 }
913 }
914 }
915 }
916 }
917 }
918 }
919 }
920 }
921 }
922 }
923 }
924 }
925 }
926 }
927 }
928 }
929 }
930 }
931 }
932 }
933 }
934 }
935 }
936 }
937 }
938 }
939 }
940 }
941 }
942 }
943 }
944 }
945 }
946 }
947 }
948 }
949 }
950 }
951 }
952 }
953 }
954 }
955 }
956 }
957 }
958 }
959 }
960 }
961 }
962 }
963 }
964 }
965 }
966 }
967 }
968 }
969 }
970 }
971 }
972 }
973 }
974 }
975 }
976 }
977 }
978 }
979 }
980 }
981 }
982 }
983 }
984 }
985 }
986 }
987 }
988 }
989 }
990 }
991 }
992 }
993 }
994 }
995 }
996 }
997 }
998 }
999 }
1000 }

```

4-track fitting

```

1 //MuMuEFPat.cc
2 void MuMuEFPat::analyze(const edm::Event& iEvent, const edm::EventSetup& iSetup)
3 {
4   //analyze
5   const MagneticField &bFieldHandle = iSetup.getData(magneticFieldToken_);
6   const double myMumass = 0.1056583745;
7   const double myMumasserr = myMumass * 1e-6; //Wonderful tools that will be used later
8   ///////////////////////////////////////////////////////////////////
9   if ( (iMuon1->charge()+iMuon2->charge()) == 0 ) {
10    TransientTrack muonPTT(muTrack1, &(bFieldHandle));
11    TransientTrack muonMTT(muTrack2, &(bFieldHandle)); //Get muon's track information, which requires track data
12    and magnetic field information, named minus
13    KinematicParticleFactoryFromTransientTrack pmumuFactory;
14    ParticleMass muon_mass = myMumass;
15    float muon_sigma = myMumasserr;
16    float chi = 0.;
17    float ndf = 0.;
18    vector< RefCountedKinematicParticle > muonParticles;
19    muonParticles.push_back(pmumuFactory.particle(muonPTT, muon_mass, chi, ndf, muon_sigma));
20    muonParticles.push_back(pmumuFactory.particle(muonMTT, muon_mass, chi, ndf, muon_sigma));
21    //The above is the fitting of track
22    KinematicParticleVertexFitter fitter;
23    RefCountedKinematicTree psiVertexFitTree;
24    psiVertexFitTree = fitter.fit(muonParticles);
25    //The above is the fitting of the vertices
26    if (psiVertexFitTree->isValid()) { //Vertex fitting is valid, then

```

```

25     psiVertexFitTree->movePointerToTheTop();//The structure of this tree is psi->mu mu, so Top is psi.
    FirstChild will be used later. NextChild refers to mu mu. Child sorting is determined according to the
    filling order of muonParticles.push_back.
26     RefCountedKinematicParticle psi_vFit_noMC = psiVertexFitTree->currentParticle();
27     RefCountedKinematicVertex psi_vFit_vertex_noMC = psiVertexFitTree->currentDecayVertex();
28     KinematicParameters mymumupara= psi_vFit_noMC->currentState().kinematicParameters();
29     //The above is to extract data from this tree
30     mumuonlyMass->push_back(psi_vFit_noMC->currentState().mass());
31
32 }
33 }// analyze

```

Try to store the following variables in Tree!

```

1 //MuMuEEPAT.cc
2 float mymumuonlyctau=GetcTau(psi_vFit_vertex_noMC,psi_vFit_noMC,theBeamSpotV);
3 float mymumuonlyctauerr=GetcTauErr(psi_vFit_vertex_noMC,psi_vFit_noMC,theBeamSpotV);
4 std::cout << "mumuonlyChg" << " " << (iMuon1->charge() + iMuon2->charge()) << std::endl;
5 std::cout << "mumuonlyctau" << " " << mymumuonlyctau << std::endl;
6 std::cout << "mumuonlyctauerr" << " " << mymumuonlyctauerr << std::endl;
7 std::cout << "mumuonlyVtxCL" << " " << ChiSquaredProbability((double)(psi_vFit_vertex_noMC->chiSquared()),
    (double)(psi_vFit_vertex_noMC->degreesOfFreedom())) << std::endl;
8 mumuonlyMass->push_back(psi_vFit_noMC->currentState().mass());
9 std::cout << "mumuonlyMass" << " " << psi_vFit_noMC->currentState().mass() << std::endl; no mass constrain
10 if( psi_vFit_noMC->currentState().kinematicParametersError().matrix()(6,6)>0) {
11     std::cout << "mumuonlyMassErr" << sqrt(psi_vFit_noMC->currentState().kinematicParametersError().matrix()
    (6,6)) << std::endl;
12 } else {
13     mumuonlyMassErr->push_back(-9);
14 }
15 std::cout << "mumuonlyPx" << " " << mymumupara.momentum().x() << std::endl;
16 std::cout << "mumuonlyPy" << " " << mymumupara.momentum().y() << std::endl;
17 std::cout << "mumuonlyPz" << " " << mymumupara.momentum().z() << std::endl;
18 std::cout << "mumuonlymulIdx" << " " << std::distance(thePATMuonHandle->begin(), iMuon1)<< std::endl;
19 std::cout << "mumuonlymu2Idx" << " " << std::distance(thePATMuonHandle->begin(), iMuon2)<< std::endl;

```

summary:

The above part can be understood as, we reconstruct the μ particles in the detector, try to fit the vertices of the reconstructed μ particles, select those processes in which two positive and negative μ particles decay from one particle, and output the physical information of the reconstructed particles.

Now you have mastered how to filter and extract the physical information of simple muon pairs in an example, which means that you have mastered the logical relationship of most of the code! The next step is to output more useful physical information, but before that, why not take a look at what your \$J/\psi\$ looks like?

5-myntuple analyzes root files

runMuMuEEPAT_data_Run2012CMSSW53XJan.py -> ivars.outputFile='mymultilep.root' will filter and output the root file based on the .cc file you wrote before. After `cmsenv` configures the environment, use the following command to run the py file

```

1 cmsRun runMultiLepPAT_dataRun3_miniAOD.py

```

A ntuple job requires myntuple.C.rootmap, myntuple.C, myntuple.h and a running Runjobs.py file

.rootmap is universal

```

1 //myntuple.C.rootmap
2 { decls }
3 [ myntuple_C.so ]
4 namespace myntuple
5 class myntuple::myntuple
6 header myntuple.h

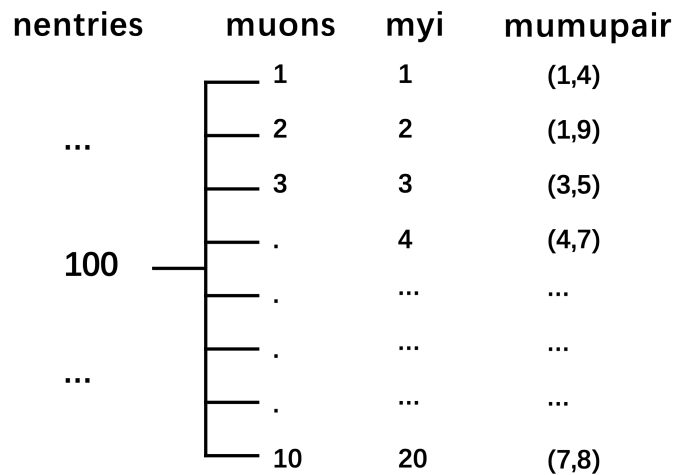
```

When you don't have .C and .h files, you can generate them through mymultilep.root. It generates a corresponding myntuple.h file and an empty myntuple.C file based on the variables in MuMuEEPat.h you edited before. Of course, you can choose your own name in `MakeClass`, but you also need to modify it in the rootmap accordingly. If you are a novice, it is best to use the same name. Note: This step will overwrite the myntuple.C and other files you generated before. You do not need to repeat this step when you have not added additional variables to MuMuEEPat.h.

```
1 $ root -l mymultilep.root
2 .ls //mkcands
3 mkcands->cd()
4 .ls //X_data
5 X_data->MakeClass("myntuple")
```

```
1 //Will generate such a .C file and a .h file containing all the variables you defined in the previous .cc
2 #define myntuple_cxx
3 #include "myntuple.h"
4 #include <TH2.h>
5 #include <TStyle.h>
6 #include <TCanvas.h>
7
8 #include <iostream>
9 void myntuple::Loop()
10 {
11     if (fChain == 0) return;
12     Long64_t nentries = fChain->GetEntries();
13     Long64_t nbytes = 0, nb = 0;
14
15     TFile* myhbk = new TFile ("myhbk.root","recreate");//Create a root and put all the histograms you want to
        draw into it
16     TH1F* mumumass = new TH1F("mumumass","mumumass",5000,0,25);//Create a one-dimensional histogram
17     TH1F* mumumasserr = new TH1F("mumumasserr","mumumasserr",5000,0,25);
18     for (Long64_t jentry=0; jentry<nentries;jentry++) { //loop for each case
19         Long64_t ientry = LoadTree(jentry);
20         if (ientry < 0) break;
21         nb = fChain->GetEntry(jentry);   nbytes += nb;
22         // if (Cut(ientry) < 0) continue;
23         for (unsigned int myi = 0; myi < mumuonlyMass->size(); myi++) { //Loop mumu in an instance
24             mumumass->Fill((*mumuonlyMass)[myi]);//Fill your variables into the histogram you defined
25             mumumasserr->Fill((*mumuonlyMassErr)[myi]);
26         }
27         if (jentry%10000 == 0) std::cout << "I am running " << jentry << " th entries out of " << nentries << " total
        entries" << std::endl;
28     }
29     myhbk->Write(); //This sentence will save all the histograms you filled in
30 } //end
```

Tips: Several concepts need to be explained here. nentries can be regarded as the number of the collision case, so the first for loop is to cycle through the collision cases used, and the second for loop is to cycle through `mumuonlyMass->size()` in one case. The number taken here is the number of the mumu pair, that is, all μ sub-pairs are cycled. Later you will also try to operate on a single μ child, and the logic is the same.



After writing myntuple.C, you need to compile it

```
1 $ root -l
2 .L myntuple.C++
```

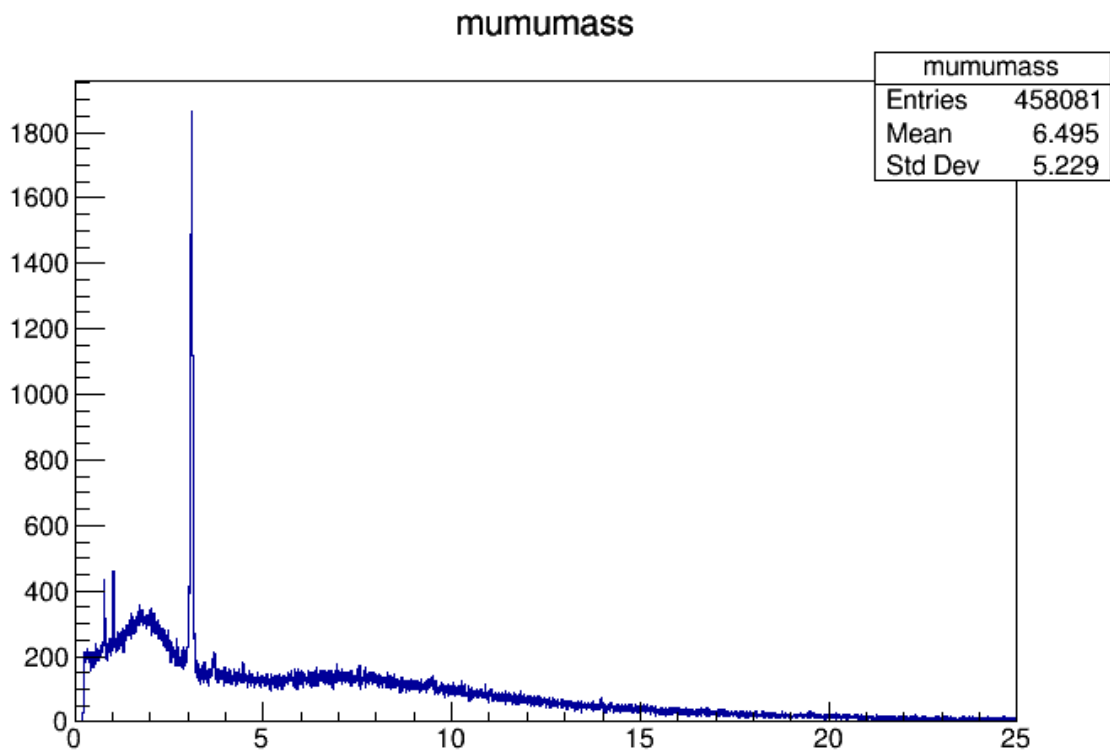
After compilation is completed, you can run Runjobs.C

```
1 void Runjobs()
2 {
3   gSystem->Load("myntuple_C.so");
4   TChain * chain = new TChain("/mkcands/X_data","");
5   chain->Add("./mymultilep.root");//Put here the file you generated in cmsRun
6   myntuple a(chain);
7   a.Loop();
8 }
```

```
1 $ root -l -b -q Runjobs.C #-l does not display the root startup interface, -b runs in batch mode and does not
display the graphical interface, -q exits after processing the command line macro file. This command is usually
used in conjunction with scripts when many files need to be run.
```

After the operation is completed, a root file will be generated, which is the myhbk.root you named in myntuple.C. You can use some commands to see the histogram inside, but it is more recommended to use vscode and the root file viewer plug-in to see it.

```
1 $ root -l myhbk.root
2 root [0]
3 Attaching file myhbk.root as _file0...
4 (TFile *) 0x1cc6e10
5 root [1] .ls
6 TFile**          myhbk.root
7 TFile*           myhbk.root
8 KEY: TH1F        mumumass;l        mumumass
9 root [2] mumumass->GetXaxis()->SetRangeUser(0.5, 5)
10 root [3] mumumass->Draw()
```



6-Output of Muon ID

Muon ID is essentially the integration of some filtering conditions of μ , labeling them with labels such as Loose, soft, medium, tight, etc. to facilitate our analysis. If you complete this chapter, you can use MuonID to set your own filtering conditions in your myntuple.C. This section will introduce the output methods of the following IDs and the necessary physical information.

```
1 muIsPatLooseMuon->push_back(iMuonP->isLooseMuon());
2 muIsPatTightMuon->push_back(iMuonP->isTightMuon(thePrimaryV));
3 muIsPatSoftMuon->push_back(iMuonP->isSoftMuon(thePrimaryV));
4 muIsPatMediumMuon->push_back(iMuonP->isMediumMuon());
```

There are two points to note:

1. Different from the previous output of mule pairs, this time the information of a single mule is output.
2. Some muon IDs require PrimaryVertex information, and PrimaryVertex needs to obtain beamSpot information.

Here are the wonderful tools you need to use, put them in the right place!

```
1 // .h
2 edm::EDGetTokenT<BeamSpot> gtbeamspotToken_;
3 edm::EDGetTokenT<VertexCollection> gtprimaryVtxToken_;
4 bool addXlessPrimaryVertex_; bool resolveAmbiguity_;
5
6 // .cc
7 MuMuEEPat::MuMuEEPat(const edm::ParameterSet& iConfig)
8 :addXlessPrimaryVertex_(iConfig.getUntrackedParameter<bool>("addXlessPrimaryVertex", true)),
9 resolveAmbiguity_(iConfig.getUntrackedParameter<bool>("resolvePileUpAmbiguity", true))
10 {
11 gtbeamspotToken_ = consumes<BeamSpot>(edm::InputTag("offlineBeamSpot"));
12 gtprimaryVtxToken_ = consumes<VertexCollection>(edm::InputTag("offlineSlimmedPrimaryVertices"));
13 }
```

Beamspot and PrimaryVertex

```
1 // .cc ->analyze
```

```

2 //Beamspt
3 Vertex theBeamSpotV;//Declare an object of type Vertex to represent a vertex
4 BeamSpot beamSpot;//Declare an object of type BeamSpot to represent the beam point
5
6 edm::Handle < reco::BeamSpot > beamSpotHandle;//Also used in the previous code to store beam information
7 iEvent.getByToken(gtbeamsptToken_, beamSpotHandle);//Extract information and store it in beamSpotHandle
8 if (beamSpotHandle.isValid()) { //If beamSpotHandle is valid, store the corresponding information
9     beamSpot = *beamSpotHandle;
10     theBeamSpotV = Vertex(beamSpot.position(), beamSpot.covariance3D());//Use beamSpot's position information
    and covariance matrix to initialize theBeamSpotV object
11 } else std::cout << "No beam spot available from EventSetup" << std::endl;
12 //Beamspt end
13
14 //PrimaryVertex
15 Vertex thePrimaryV;
16 math::XYZPoint RefVtx;//Used to represent three-dimensional points or vectors, store PrimaryVertex
17
18 edm::Handle < VertexCollection > recVtxs;
19 iEvent.getByToken(gtprimaryVtxToken_, recVtxs);
20
21 unsigned int nVtxTrks = 0;//Unsigned integers are positive integers to avoid negative values
22 int mynGoodPrimVtx=0;
23
24 for(unsigned myi=0;myi<recVtxs->size();myi++) { //Traverse all vertices obtained by recVtxs
25     if((*recVtxs)[myi].ndof()>=5 && fabs((*recVtxs)[myi].z())<=24 && fabs((*recVtxs)[myi].position().rho())
    <=2.0) { //The vertex conditions include ndof degrees of freedom, z coordinates, and rho radial distance
26         mynGoodPrimVtx++; //The qualified one is goodPrimaryVertex, count +1
27     }
28 }
29 nGoodPrimVtx = mynGoodPrimVtx;//Remember to output this variable
30
31 if (recVtxs->begin() != recVtxs->end()) { //Check if it is empty
32     if (addXlessPrimaryVertex_ || resolveAmbiguity_) { //If true, select the first vertex as thePrimaryV
33         thePrimaryV = Vertex(*recVtxs->begin());
34     } else { //If not true, select the vertex with the largest number of trajectories as thePrimaryV
35         for (reco::VertexCollection::const_iterator vtx = recVtxs->begin(); vtx != recVtxs->end(); ++vtx) {
36             if (nVtxTrks < vtx->tracksSize()) {
37                 nVtxTrks = vtx->tracksSize();
38                 thePrimaryV = Vertex(*vtx);
39             }
40         }
41     }
42 } else { //If it is empty, initialize it with the previous beamspt information
43     thePrimaryV = Vertex(beamSpot.position(), beamSpot.covariance3D());
44 } //This part ensures that there is a reasonable main vertex available for use in vertex reconstruction and
    physical analysis
45
46 //Next export the position information of the initial vertex
47 RefVtx = thePrimaryV.position();
48 priVtxX = (thePrimaryV.position().x());
49 priVtxY = (thePrimaryV.position().y());
50 priVtxZ = (thePrimaryV.position().z());
51 priVtxXE = (thePrimaryV.xError());
52 priVtxYE = (thePrimaryV.yError());
53 priVtxZE = (thePrimaryV.zError());
54 priVtxChiNorm = (thePrimaryV.normalizedChi2());
55 priVtxChi = thePrimaryV.chi2();
56 priVtxCL = ChiSquaredProbability((double) (thePrimaryV.chi2()), (double) (thePrimaryV.ndof()));
57 //PrimaryVertex end

```

Muon ID and Muon block

```

1 //cc ->If you have fully mastered the more complicated situations before, this next part should be very
    simple for you :)

```

```

2 edm::Handle< edm::View<pat::Muon> > thePATMuonHandle;//Similarly define the handle and then extract the
  information
3 iEvent.getByToken(gtpatmuonToken_, thePATMuonHandle);
4 edm::View<pat::Muon>::const_iterator iMuonP;
5 for (iMuonP = thePATMuonHandle->begin(); iMuonP != thePATMuonHandle->end(); ++iMuonP) {
6     ++nMu;//muon counter, used for subsequent efficiency calculations
7     muIsPatLooseMuon->push_back(iMuonP->isLooseMuon());
8     muIsPatTightMuon->push_back(iMuonP->isTightMuon(thePrimaryV));
9     muIsPatSoftMuon->push_back(iMuonP->isSoftMuon(thePrimaryV));
10    muIsPatMediumMuon->push_back(iMuonP->isMediumMuon());
11    muPx->push_back(iMuonP->px());
12    muPy->push_back(iMuonP->py());
13    muPz->push_back(iMuonP->pz());
14    muCharge->push_back(iMuonP->charge());
15 }

```

7-Processing data

If everything goes well, you should see a very sharp peak near 3Gev in the mass spectrum of mumu. Yes, this is the J/ψ particle you reconstructed! Congratulations! But at a glance like this, I can tell that the peak I found is the J/ψ peak of 3.0969Gev. It will obviously be beaten to death by Mr. Ding Zhaozhong. We also need to fit these data points and mathematically determine what particle's decay function the peak we find is.

```

1 TH1F* mumumass = new TH1F("mumumass","mumumass",5000,0,25);//5000 is the number of bins

```

Remember the one-dimensional histogram named mumumass we defined in the myntuple.C file before, where we set its range of 0-25 and a bin of 5000. The bin here can be understood as the pillars of a bar graph, which means that we will lose some fine data. For example, we cannot distinguish two mumu with masses of 3.000 and 3.0002, because they will be thrown into the bin of 3.000~3.005. Of course, if you have enough bins, you can also call it "fine", which is also the method of calculus. But why not directly get the mass value of each mumu pair! Therefore, although the histogram you output here can be used for fitting, if you want to be more free, you need to output the quality of each mumu pair, so that you can change your bin as you like in your fitting program!

It is relatively simple to output the txt part. You can output it directly in myntuple.C using the following method. I believe you should be able to put these two puzzle pieces in the right place. Remember to add spaces between the two variables when outputting multiple variables. This way, when you need to input multiple variables later, you can divide them into different columns by spaces.

```

1 fstream myoutfile;
2 myoutfile.open("mydata.txt",std::ios::out);//Place outside the loop
3 ...
4 myoutfile << std::fixed
5     << (*mumuonlyMass)[myi] << " " << (*mumuonlyMassErr)[myi]
6     << std::endl;
7 //Put it inside the loop

```

After you successfully output a txt file, you can create a new fitting program.

The fitting part is officially carried out below, and if you have any questions about the variables used that I have not included, you can find the answer in this link [ROOT: RooFit \(cern.ch\)](#), you can also refer to my other note to learn more about the usage of RooFit -> [roofit manual](#)

For example, you can find the usage of `RooRealVar`. If you make good use of the search function, you can also become a master!

```

RooRealVar::RooRealVar ( const char * name,
                        const char * title,
                        double      minValue,
                        double      maxValue,
                        const char * unit = ""
                      )

```

```

1  #include <TFile.h>
2  #include <TH1.h>
3  #include "RooAbsReal.h"
4  #include <RooRealVar.h>
5  #include <RooDataHist.h>
6  #include <RooGaussian.h>
7  #include <RooPlot.h>
8  #include <RooFitResult.h>
9  #include "RooChebychev.h"
10 #include "RooAddPdf.h"
11 #include "TCanvas.h"
12 #include "RooCrystalBall.h"
13 #include <RooFit.h>
14 using namespace RooFit;
15 using namespace std;
16 void fitroot() { //This needs to be the same as your file name
17     RooRealVar mass("mass", "Invariant Mass", 0.5, 4); //Define variables, which is the most commonly used
18     RooArgSet variables;
19     variables.add(mass); //Created a variable set and added the variable mass in it
20
21     RooDataSet *data = RooDataSet::read("./mydata.txt", variables, "Q"); //Read the data. When you need to
22     //input multiple variables, please make sure that the order of your variable set is consistent with the order
23     //of your output in mytuple.C. That is to say, your variables need to correspond to each column (separated by
24     //spaces) in your txt file.
25     mass.setBins(300); //This is not to bin the data, but to bin the variables, which is equivalent to
26     //sorting some empty boxes first.
27     RooDataHist datahist("datahist", "binned data", RooArgSet(mass), *data); //This step is to divide the
28     //data into the previous boxes
29     //The following can be ignored
30     //TFile *file = TFile::Open("myhbk.root");
31     //TH1F *hist = (TH1F*)file->Get("mumumassSoft"); //These two steps are to extract your histogram directly
32     //from the root
33     //TH1F *rebinHist = (TH1F*)hist->Rebin(2, "rebinHist"); //This means merging the two bins of the
34     //histogram into one
35     //RooDataHist data("data", "Dataset with mass", mass, hist);
36     //
37
38     RooRealVar c0("c0", "c0", 0.5, -1., 1.);
39     RooRealVar c1("c1", "c1", 0.1, -1., 1.);
40     RooRealVar c2("c2", "c2", 0.1, -1., 1.);
41     RooRealVar c3("c3", "c3", 0.1, -1., 1.);
42     RooChebychev chev("chev", "chev", mass, RooArgList(c0, c1, c2, c3)); //Chebyshev polynomials
43
44
45     RooRealVar psimean("psimean", "Mean", 3.1, 2.9, 3.3);
46     RooRealVar psisigma("psisigma", "Sigma", 0.05, 0.001, 0.1);
47     RooRealVar psialpha("psialpha", "psialpha", 1., 0.0, 2.0);
48     RooRealVar psin("psin", "psin", 1., 0, 15);
49     RooCrystalBall psicb("psicb", "psicb", mass, psimean, psisigma, psialpha, psin, false); //Crystal Ball function
50     //RooGaussian psigauss("psigauss", "Gaussian PDF", mass, psimean, psisigma);
51
52     RooRealVar njpsi("njpsi", "signal fraction", 500, 0., 10000.);
53     RooRealVar nbkg("nbkg", "background fraction", 500, 0., 10000.);
54
55     RooExtendPdf epsisig("esisig", "esisig", psicb, njpsi);
56     RooExtendPdf epsibkg("ebkg", "ebkg", chev, nbkg); //This part has some knowledge prerequisites. In short,
57     //the number of components can be obtained directly after extending pdf
58
59     RooAddPdf model("fsig", "fsig", RooArgList(epsisig, epsibkg)); //model(x) = (njpsi/(njpsi+nbkg))*psicb(x) +
60     //(1-(nbkg/(njpsi+nbkg)))*bkg(x)
61     RooFitResult* result = model.fitTo(*data, Save());
62     //RooFitResult* result = model.fitTo(datahist, Save());
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

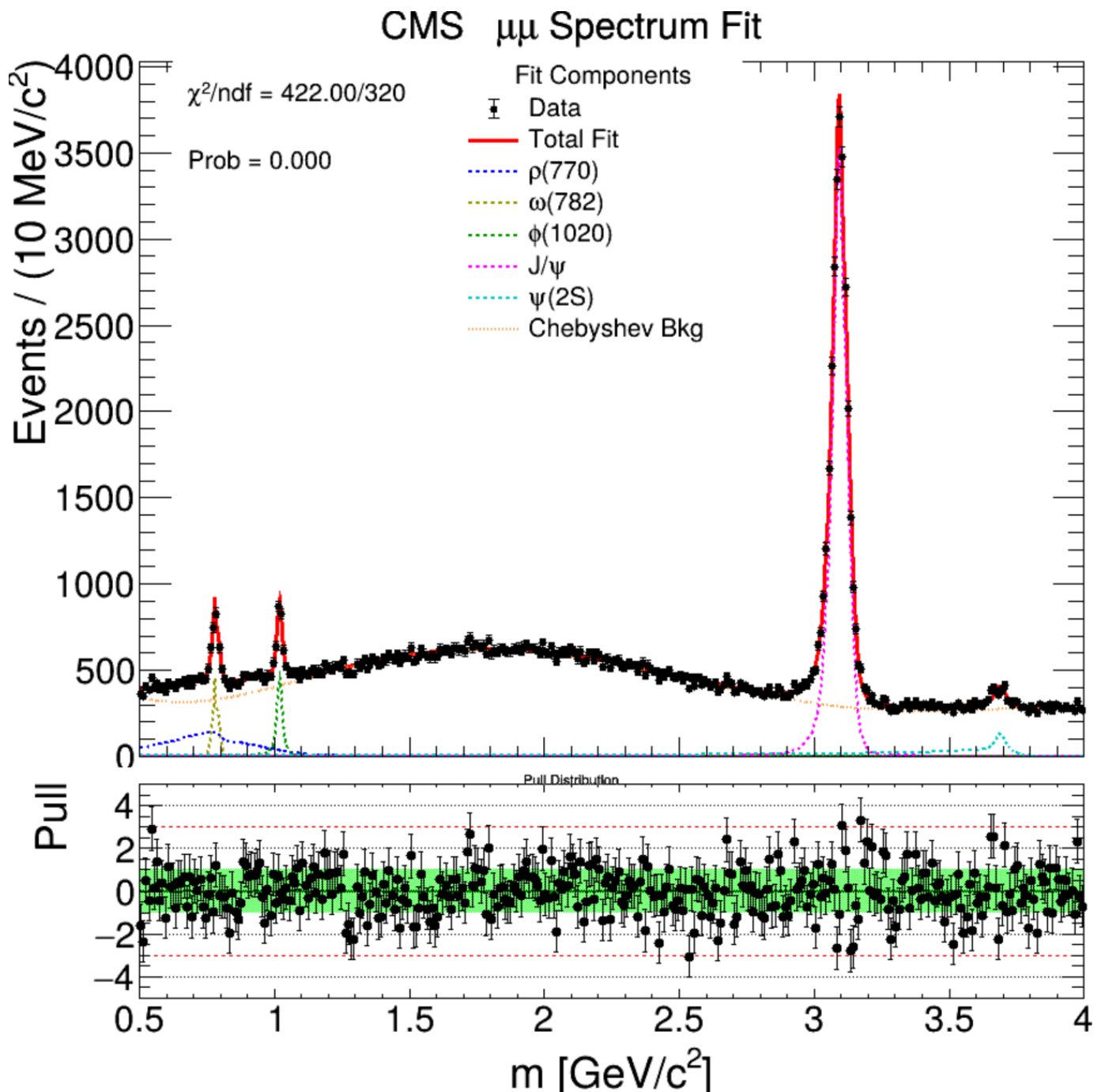
```

```

56 RooPlot* frame = mass.frame();//Create a coordinate axis with mass equivalent to the x-axis
57 data->plotOn(frame,MarkerStyle(20),MarkerSize(0.4),Name("data"));
58 //datahist.plotOn(frame,MarkerStyle(20),MarkerSize(0.4),Name("data"));
59 model.plotOn(frame,LineColor(2),LineWidth(3),Name("model"));
60 model.plotOn(frame,Components(epsisig),LineColor(7),LineStyle(1),LineWidth(3),Name("Jpsi"));
61 model.plotOn(frame,Components(epsibkg),LineColor(9),LineStyle(1),LineWidth(3),Name("BKG"));
62 //Draw the corresponding part onto this coordinate axis
63
64 TLegend yleg(0.7,0.7,0.9,0.9);//Draw a rectangle starting from (0.7,0.7) position relative to the lower
left corner of a canvas to (0.9,0.9)
65 yleg.AddEntry(frame->findObject("model"),"FullModel","L");
66 yleg.AddEntry(frame->findObject("Jpsi"),"Jpsi","L");
67 yleg.AddEntry(frame->findObject("BKG"),"Background","L");
68 //Add legend
69
70 RooPlot *xfpull=mass.frame();//Draw pull distribution
71 RooHist *pullx=frame->pullHist("data","model");//The deviation of data points from the fitted model
graph line
72 xfpull->addPlotable(pullx,"p");
73 //xfpull->GetXaxis()->SetTitle("J/#psiJ/#psi");
74 xfpull->GetXaxis()->SetTitleSize(0.15);
75 xfpull->GetXaxis()->SetLabelSize(0.12);
76 xfpull->GetYaxis()->SetTitle("Pull");
77 xfpull->GetYaxis()->SetTitleSize(0.15);
78 xfpull->GetYaxis()->SetLabelSize(0.1);
79 xfpull->GetYaxis()->SetTitleOffset(0.2);
80
81 TCanvas c ("c", "Fit with RooFit", 800, 600);//Create a canvas
82 c.cd();
83 TPad pad11("pad11","pad11",0,0.3,1,1.0);//Create the range from (0,0.3) to (1,1)
84 pad11.SetTopMargin(0.08);
85 pad11.SetBottomMargin(0.017);
86 pad11.Draw();
87 pad11.cd();//Enter this area to draw the plot and legend you want to draw.
88 frame->Draw();
89 yleg.Draw();
90 c.cd();
91 TPad pad12("pad12","pad12",0,0.0,1,0.3);
92 pad12.SetTopMargin(0.03);
93 pad12.SetBottomMargin(0.325);
94 pad12.SetGridx();
95 pad12.SetGridy(2);
96 pad12.Draw();
97 pad12.cd();
98 xfpull->Draw();
99 c.Update();
100 c.cd();
101
102 c.SaveAs("fitResult.pdf");
103
104 }
105

```

If nothing else, you will end up with a very nice fitting graph. Maybe you will find some smaller peaks, they are $\rho(770)$, $\omega(782)$, ϕ , $\psi(2S)$, then try to find the mass of these particles on [PDG](#), and use the `RooBreitWigner` or `RooGaussian` function to fit these peaks! I'm sure you'll get a result that looks great!



Now take a break with a Coke! The content behind will be more complicated!

pre 1-CRAB jobs submit

So far, you have most likely only used one root file I put in the folder. Although it is enough for the current exercise to obtain considerable results, as the filtering conditions and analysis requirements increase, one root file may not be able to meet our needs, so many root files need to be processed in batches. However, this processing does not need to be performed locally. Instead, a "job" is submitted, allowing computing clusters distributed around the world to complete the tasks you need to process, and then store it on the server you specify (this is cool!). This is the CRAB (CMS Remote Analysis Builder) of CMS.

Next, I will teach you how to submit a crab job. Please note that **CRAB relies on your certificate**. All the following operations are based on the fact that you already have the certificate and use the `voms-proxy-init --voms cms` command in the terminal to obtain the certificate agent.

- [DAS\(Data Aggregation System\)](#)

First you need to find the data tags you want to process. In fact, CMS collision data will be reconstructed and classified multiple times, each time corresponding to a data label. As for which data label you need to use in your research, you need to confirm with the corresponding personnel. Then you can check all the data you need on the DAS website. For example, for the `/ParkingDoubleMuonLowMass*/Run2024*-PromptReco-v1/MINIAOD` label, you can see that there are 64 data labels corresponding to it. Each label corresponds to the data running in a certain period of time. These are what you need to process.


Data Aggregation System (DAS):
[Home](#) | [Services](#) | [Keys](#) | [Bug report](#) | [Status](#) | [CLI](#) | [FAQ](#) | [Help](#)

results format: , results/page, dbs instance ,

[Show DAS keys description](#)

Showing 1 — 50 records out of 64.

By default DAS shows dataset with **VALID** status. To query datasets regardless of their status please use

```
dataset status=* dataset=/ParkingDoubleMuonLowMass*/Run2024*-PromptReco-v1/MINIAOD
```

Dataset: [/ParkingDoubleMuonLowMass0/Run2024B-PromptReco-v1/MINIAOD](#)
 Creation time: 2024-04-08 18:23:22 Cross section: 0 Physics group: NoGroup Status: **VALID** Type: data
[Release](#), [Blocks](#), [Files](#), [Runs](#), [Configs](#), [Parents](#), [Children](#), [Sites](#), [Physics Groups](#) [XSDB](#) Sources: [dbs3](#) [show](#)

Dataset: [/ParkingDoubleMuonLowMass0/Run2024C-PromptReco-v1/MINIAOD](#)
 Creation time: 2024-04-16 20:14:39 Cross section: 0 Physics group: NoGroup Status: **VALID** Type: data
[Release](#), [Blocks](#), [Files](#), [Runs](#), [Configs](#), [Parents](#), [Children](#), [Sites](#), [Physics Groups](#) [XSDB](#) Sources: [dbs3](#) [show](#)

Dataset: [/ParkingDoubleMuonLowMass0/Run2024D-PromptReco-v1/MINIAOD](#)
 Creation time: 2024-05-04 22:45:05 Cross section: 0 Physics group: NoGroup Status: **VALID** Type: data
[Release](#), [Blocks](#), [Files](#), [Runs](#), [Configs](#), [Parents](#), [Children](#), [Sites](#), [Physics Groups](#) [XSDB](#) Sources: [dbs3](#) [show](#)

Dataset: [/ParkingDoubleMuonLowMass0/Run2024E-PromptReco-v1/MINIAOD](#)
 Creation time: 2024-05-24 00:17:31 Cross section: 0 Physics group: NoGroup Status: **VALID** Type: data
[Release](#), [Blocks](#), [Files](#), [Runs](#), [Configs](#), [Parents](#), [Children](#), [Sites](#), [Physics Groups](#) [XSDB](#) Sources: [dbs3](#) [show](#)

Dataset: [/ParkingDoubleMuonLowMass0/Run2024F-PromptReco-v1/MINIAOD](#)
 Creation time: 2024-06-21 16:55:02 Cross section: 0 Physics group: NoGroup Status: **VALID** Type: data
[Release](#), [Blocks](#), [Files](#), [Runs](#), [Configs](#), [Parents](#), [Children](#), [Sites](#), [Physics Groups](#) [XSDB](#) Sources: [dbs3](#) [show](#)

Doesn't it seem like a lot? Do you still remember the root file I gave you? Where is its location? You can click the [File](#) button below. Each data label contains root files. The number and size will be different. For example, the label below contains 894 root files. If the root files in all labels are added together, it must be a very large number.

results format: , results/page, dbs instance ,

[Show DAS keys description](#)

Showing 1 — 50 records out of 894.

File name: [/store/data/Run2024H/ParkingDoubleMuonLowMass3/MINIAOD/PromptReco-v1/000/385/836/00000/0b80e21c-c3b4-4976-a9e2-a2c839cd6d08.root](#)
 File size: 2.552272661e+09 (2.6GB) File size: 2552272661 (2.6GB) File type: EDM Number of events: 44239
[Dataset](#), [Block](#), [Sites](#), [Runs](#), [Parents](#), [Children](#), [Lumis](#) Sources: [dbs3](#) [rucio](#) [show](#)

File name: [/store/data/Run2024H/ParkingDoubleMuonLowMass3/MINIAOD/PromptReco-v1/000/385/836/00000/1d4819bf-4083-4c1f-b8cd-7f088460d698.root](#)
 File size: 8.43176008e+08 (843.2MB) File size: 843176008 (843.2MB) File type: EDM Number of events: 14078
[Dataset](#), [Block](#), [Sites](#), [Runs](#), [Parents](#), [Children](#), [Lumis](#) Sources: [dbs3](#) [rucio](#) [show](#)

File name: [/store/data/Run2024H/ParkingDoubleMuonLowMass3/MINIAOD/PromptReco-v1/000/385/836/00000/388c6bec-1858-48cf-bfc1-f2c4a18b1483.root](#)
 File size: 4.040535215e+09 (4.0GB) File size: 4040535215 (4.0GB) File type: EDM Number of events: 67995
[Dataset](#), [Block](#), [Sites](#), [Runs](#), [Parents](#), [Children](#), [Lumis](#) Sources: [dbs3](#) [rucio](#) [show](#)

If you want to test other root files, you can copy the root data file on DAS to the local computer:

```
xrdcp -d 1 -f root://xrootd-cms.infn.it//store/data/Run2024H/ParkingDoubleMuonLowMass3/MINIAOD/PromptReco-v1/000/385/836/00000/0b80e21c-c3b4-4976-a9e2-a2c839cd6d08.root .
```

You can also put this address directly in the py file. Multiple ROOT files need to be connected (a certificate is required for this) using `,`.

```
1 ivars.inputFiles=( 'root://xrootd-cms.infn.it//store/data/Run2024H/ParkingDoubleMuonLowMass3/MINIAOD/PromptReco-v1/000/385/836/00000/0b80e21c-c3b4-4976-a9e2-a2c839cd6d08.root' ,
2                      'root://xrootd-cms.infn.it//store/data/Run2024I/ParkingDoubleMuonLowMass0/MINIAOD/PromptReco-v1/000/386/478/00000/1b1f1417-5da2-40be-b5da-3ab3daaf52cb.root' )
```

The information in other tags is also very important. Here I will briefly introduce what we can use:

- `release` CMSSW version information. Generally, we will use the latest CMSSW version corresponding to this data tag. As I explained in the preface, there will be many different CMSSW versions every year, so corresponding needs to be made.

results format: , results/page, dbs instance ,

release dataset=/ParkingDoubleMuonLowMass0/Run2024B-PromptReco-v1/MINIAOD

[Show DAS keys description](#)

Showing 1 — 1 records out of 1.

Release name: **CMSSW 14_0_4**
[Datasets](#) Sources: **dbs3** [show](#)

Showing 1 — 1 records out of 1.

- `config` Here we usually get the Global Tag (GT) corresponding to the data, which is equivalent to a "configuration tag", used to tell CMSSW which set of calibration, alignment, detector status and other condition information should be used when running. This version will continue to change, and like CMSSW, use the latest GT version corresponding to this data tag. GT needs to be modified in the py file of cmsRun. Find the following part of the py file. If is the GT of MC, and else is the GT of data, which corresponds to the GT of modified data.

```
1 if runOnMC:
2     process.GlobalTag.globaltag = cms.string( 'MCRUN2_74_V9::A11' )
3 else:
4     process.GlobalTag = GlobalTag(process.GlobalTag, '132X_dataRun3_Prompt_v4', '')
```

results format: , results/page, dbs instance ,

config dataset=/ParkingDoubleMuonLowMass0/Run2024B-PromptReco-v1/MINIAOD

[Show DAS keys description](#)

Showing 1 — 1 records out of 1.

Request name: cmsRun
Created by: /DC=ch/DC=cern/OU=Organic Units/OU=Users/CN=cmst0/CN=658085/CN=Robot: CMS Tier0 Creation time: 2024-04-02 23:05:39 Global Tag: 140X_dataRun3_Prompt_v2 Pset hash: GIBBERISH Release: CMSSW_14_0_4 Request urls:
[ReqMgr info](#) Sources: **dbs3** [show](#)

Showing 1 — 1 records out of 1.

• CRAB configuration file

After the above query and confirmation, and the debugging of the PAT package has been completed, then you need to edit a crab configuration file to submit the crab job. Next, I will introduce the meaning of these 6 places that need to be modified:

```

1 from WMCore.Configuration import Configuration
2 config = Configuration()
3
4 config.section_('General')
5 config.General.transferOutputs = True
6 config.General.requestName = '2024JEta_Run2024I-PromptReco-v1' #modify 1
7
8 config.section_('JobType')
9 config.JobType.psetName = './runMultiLepPAT_dataRun3_miniAOD.py' #modify 2
10 config.JobType.pluginName = 'Analysis'
11 config.JobType.outputFiles = ['mymultilep.root']
12 config.JobType.allowUndistributedCMSSW = True
13
14 config.section_('Data')
15 config.Data.inputDataset = '/ParkingDoubleMuonLowMass0/Run2024I-PromptReco-v1/MINIAOD' #modify 3
16 config.Data.inputDBS = 'global'
17 config.Data.unitsPerJob = 50
18 config.Data.splitting = 'LumiBased'
19 #config.Data.outLFNDirBase = '/store/group/lpcbphy/noreplica/miniAOD_zhuf/JEta_July22/'
20 config.Data.outLFNDirBase = '/store/user/zhuf/JEta_July22/' #modify 4
21 config.Data.outputDatasetTag = '2024JEta_Run2024I-PromptReco-v1' #modify 5
22
23 config.section_('User')
24 config.section_('Site')
25 config.Site.storageSite = 'T3_CH_CERNBOX' # T3_US_FNALLPC #modify 6

```

- **M1: `config.General.requestName`**
 - This is the name of the submitted job (task name). This becomes the unique ID that CRAB recognizes on the server and in the database.
 - It is generally recommended to include **physical process/data set name + version number/date** to facilitate distinguishing different submissions.
 - It cannot be repeated, otherwise CRAB will report an error (even if you delete the local task directory, there will still be a conflict on the server), so you need to edit the corresponding crab config file for each data label.
 - **M2: `config.JobType.psetName`**
- Specify the CMSSW profile (`pset`) you want to run. Usually it is a `.py` file, such as `runMultiLepPAT_dataRun3_miniAOD.py` here, and the relative/absolute path of this file needs to be provided.
 - **M3: `config.Data.inputDataset`**
- This is the official dataset name you want to process (obtained from the DAS query).
 - **M4: `config.Data.outLFNDirBase`**
- Specify your **output file saving path** (under storage systems such as CERN EOS or FNAL dCache). Generally use `/store/user/<username>/...`. You need to write a subfolder here to differentiate between different analyses.
 - **M5: `config.Data.outputDatasetTag`**
- This is the label of the output dataset, similar to a suffix. It will be appended to the name of your output dataset to make it easier to distinguish it in DAS. Generally consistent with `requestName`, or slightly simplified.
- If you submit the same crab repeatedly, the output file returned by the crab will generate a folder corresponding to the time in the same path according to the time you submitted. The path of the generated output folder is similar to

```

1 `outLFNDirBase`+`inputDataset(1)`+`requestName` :
2
3 `/store/user/<username>/JEta_July22/ParkingDoubleMuonLowMass0/2024JEta_Run2024I-PromptReco-
  v1/250602_020604`

```

- **M6: `config.Site.storageSite`**

- Specify which **storage site** (SE, Storage Element) your output data should be written to. For example: `T3_CH_CERNBOX`, `T3_US_FNALLPC`
- You need to select a site that you have write permission for, otherwise an error will be reported when submitting. Below are some commonly used sites and commands for checking permissions. In fact, under normal circumstances, you have permission, but it may happen that you take up too much space, and the corresponding administrator will find you and ask you to delete the unused ones, so you need to be proficient in the following check and remove operations.

```

1  T3_CH_CERNBOX
2  T2_CN_Beijing
3
4  T3_US_FNALLPC
5
6  cmsenv
7  voms-proxy-init --rfc --voms cms
8  crab checkwrite --site=T2_CN_Beijing
9  which scram >/dev/null 2>&1 && eval `scram unsetenv -sh` (this step usually finds the path prefix used
   by the commands below: davs://cceos.ihep.ac.cn:9000/eos/ihep/cms/store/user/zhuf/)
10
11  ```shell
12  gfal-ls -lH davs://cceos.ihep.ac.cn:9000/eos/ihep/cms/store/user/zhuf/`
13  gfal-rm -r 'davs://cceos.ihep.ac.cn:9000/eos/ihep/cms/store/user/zhuf/SPStosingleJ_2023HI/'
14  ```

```

One sentence summary:

- M1 is the name of the job
- M2 is the running configuration file
- M3 is the input data set
- M4 is the output file storage path
- M5 is the output dataset label
- M6 is the storage site

• CRAB submission, status query and resubmission

After you edit your config file and name it `crab3_XXX.py`, you can use `crab submit crab3_XXX.py` to submit your CRAB job. The correct output is similar to the following:

```

• test$crab submit crab3_2024Jpipi.py
Will use CRAB configuration file crab3_2024Jpipi.py
Importing CMSSW configuration ./runMultiLepPAT_dataRun3_miniaOD.py
Finished importing CMSSW configuration ./runMultiLepPAT_dataRun3_miniaOD.py
Warning: The following output files will not be published, as they are not EDM files: ['mymultilep.root']
Sending the request to the server at cmsweb.cern.ch
Success: Your task has been delivered to the prod CRAB3 server.
Task name: 250831_142639:zhuf_crab_2024Jpipi_Run2024I-PromptReco-v1
Project dir: ./crab_2024Jpipi_Run2024I-PromptReco-v1
Please use 'crab status -d ./crab_2024Jpipi_Run2024I-PromptReco-v1' to check how the submission process
proceeds.
Log file is /uscms_data/d3/fengzhu/Mar6_FourMuTwoPion/CMSSW_14_0_16/src/UserCode/FourMuTwoPion/test/crab_2
024Jpipi_Run2024I-PromptReco-v1/crab.log

```

Then a folder named `crab_+requestName` will be generated where you submitted the crab. You can use `crab status crab_2024Jpipi_Run2024I-PromptReco-v1` to query the status of your job:

```
fengzhu@cmslpc312:~/nobackup/Mar6_FourMuTwoPion/CMSSW_14_0_16/src/UserCode/FourMuTwoPion/test$crab status
-d ./crab_2024Jpipi_Run2024I-PromptReco-v1
Rucio client intialized for account zhuf
CRAB project directory: /uscms_data/d3/fengzhu/Mar6_FourMuTwoPion/CMSSW_14_0_16/src/UserCode/FourMuTwoPion/test/crab_2024Jpipi_Run2024I-PromptReco-v1
Task name: 250831_142639:zhuf_crab_2024Jpipi_Run2024I-PromptReco-v1
Grid scheduler - Task Worker: crab3@vocms0196.cern.ch - crab-prod-tw01
Status on the CRAB server: SUBMITTED
Task URL to use for HELP: https://cmsweb.cern.ch/crabserver/ui/task/250831_142639%3Azhuf_crab_2024Jpipi_Run2024I-PromptReco-v1
Dashboard monitoring URL: https://monit-grafana.cern.ch/d/cmsTMDetail/cms-task-monitoring-task-view?orgId=11&var-user=zhuf&var-task=250831_142639%3Azhuf_crab_2024Jpipi_Run2024I-PromptReco-v1&from=1756646799000&to=now
Status on the scheduler: SUBMITTED
Jobs status: unsubmitted 100.0% (1405/1405)
No publication information available yet
Log file is /uscms_data/d3/fengzhu/Mar6_FourMuTwoPion/CMSSW_14_0_16/src/UserCode/FourMuTwoPion/test/crab_2024Jpipi_Run2024I-PromptReco-v1/crab.log
```

```
Status on the scheduler: SUBMITTED
Jobs status: failed 63.8% ( 897/1405)
idle 16.1% ( 226/1405)
running 13.1% ( 184/1405)
toRetry 7.0% ( 98/1405)
No publication information available yet
Error Summary: (use crab status --verboseErrors for details about the errors)
897 jobs failed with exit code 8001
Have a look at https://twiki.cern.ch/twiki/bin/viewauth/CMSPublic/JobExitCodes for a description of the exit codes.
```

We usually pay attention to the status in the red box, where `Status on the scheduler` is the general status of the job, `Jobs status` is some specific status, `finished` refers to success, `failed` refers to failure (nonsense), and the others are some intermediate statuses that are not very important. Of course, success may not be very important. The main thing is to solve the possible `failed` status. You can make some attempts like this (you need to use crab below) I made a simple replacement in the folder):

- `crab resubmit crab_xx` Occasionally, several files fail, which is usually a problem with certain sites. In this case, you only need to resubmit and let the site run again. Multiple resubmits may solve the problem.
- Query exit codes. According to the output, you can see that it will return an error code. You can find the corresponding code in the URL it prompts, and you can roughly understand the cause of the error.

Error codes currently sent from CMS jobs to the dashboard

⚠ indicates site error

- **Error exit code of the cmsRun application itself - range 0-10000**
- **Exit codes in 1-255 are standard ones in Unix and indicate a CMSSW abort that the cmsRun did not catch as exception**
 - 1 - 255 those exit codes are usually returned by `bash` when it is interrupted by a fatal signal. The exit code is set to `signal`
 - For convenience we have collected a list of most common such exit codes: [StandardExitCodes](#)
- **cmsRun (CMSSW) exit codes. range 7000-9000 * *These codes may depend on specific CMSSW version, the list**
 - // The first three are specific categories of CMS Exceptions.
 - 7000 - Exception from command line processing
 - 7001 - Configuration File Not Found
 - 7002 - Configuration File Read Error
 - 8001 - Other CMS Exception
 - 8002 - std::exception (other than bad_alloc)
 - 8003 - Unknown Exception
 - 8004 - std::bad_alloc (memory exhaustion)
 - 8005 - Bad Exception Type (e.g throwing a string)
- Query the log in the Dashboard. There will be a `Dashboard monitoring URL` in the line above the red box in CRAB status. This URL displays the complete log of your CRAB task, and the JobLog contains the log output during runtime.

For example, after I went through the previous steps and found that they were useless, I knew there was a problem with my program.

```
1 ! [image-20250831225541433] (/postfigure/multilepnote/image-20250831225541433.png)
2
3 ! [image-20250831225714494] (/postfigure/multilepnote/image-20250831225714494.png)
```

Now that I know it is a problem with my program, when I modify my program and need to resubmit it, I need to pay attention to these issues:

- Clean up possible previous output files. Although crab will have a folder divided according to the submission time when outputting, it is still recommended to delete the previous output when resubmitting, not only for storage space considerations, but also for data duplication caused by subsequent processing.
- Delete the crab_xx folder that was automatically generated when submitting CRAB. If you do not do this step, you will be prompted that this folder already exists and cannot be submitted. Of course, you can also generate different crab_xx folders by modifying `requestName`.
- Use `crab submit crab3_xxx.py` again to submit your CRAB task. Note that unlike the `crab resubmit` command, resubmit will not use your latest files because you have modified your program or other configurations.

CRAB is a very useful tool. After you make sure that your program has no problems, you will feel smooth when submitting CRAB, but you should also pay attention to whether your storage space is enough!

pre 2-Condor Jobs submit

When you submit a lot of CRAB, a lot of root files will be output. When doing the next step of processing, you will also encounter the dilemma that there are too many and cannot be processed all at once. Then you need to submit Condor Jobs for the computer to process in parallel. It should be noted that Condor is equivalent to running many programs in the background at the same time. You need to consider whether there will be conflicts between these programs because they output files with the same name. You also need to consider the balance between the number of submitted Condor Jobs and the size of the files processed by each job - the more detailed you divide the task, the faster the processing time of a single task, but if a simple task is divided into small pieces, it will increase the time it takes for the entire task to be completed.

Unlike CRAB, which can call global computing resources, Condor uses the computing resources of the current cluster, and the files he can input can only be files on the current cluster. Writing the files required for Condor submission is actually more like writing a script for automated processing, so writing a script that can be fully automated is the most important. I will use a commonly used example to illustrate below. If you don't understand the following, you can refer to the [this guide](#) link for a more complete learning process.

We often submit condor tasks to batch process tasks that need to be processed using the myntulpe.C program. Before submitting condor, you need to prepare all the program files to be used, which are mainly divided into myntulpe.C and runjobs.C program related, automated script Run.sh, and condor jdl (job description language) file.

The way of using CMSSW in Run.sh is different from the way of using it locally, but the logic is the same. All the output of my run jobs.C is placed in the output folder, so I need to use `mkdir output` in the script to create this folder and then run the subsequent program.

```
1 #!/bin/bash
2 source /cvmfs/cms.cern.ch/cmsset_default.sh
3 export SCRAM_ARCH=e19_amd64_gcc12
4 eval `scramv1 project CMSSW CMSSW_14_0_4`
5 cd CMSSW_14_0_4/src/
6 eval `scramv1 runtime -sh`
7 cd ../../
8 mkdir -p output
9 root -b -q -l runjobs.C
```

The following is the condor jdl file `runOnCondor`, which indicates the file to run the script `Executable = Run.sh`, the file uploaded to the condor `Transfer_Input_Files`, the file transferred to the local `transfer_output_files`, and the log file output location when the condor is running `Output` `Error` `Log`. Here I output all to the `logfile` folder, so I also need to first create the local `mkdir logfile` folder.

```

1 universe = vanilla
2 Executable = Run.sh
3 Should_Transfer_Files = YES
4 Transfer_Input_Files = "myntulpe.C, myntulpe.C.rootmap, myntulpe.h, myntulpe.C.d, myntulpe.C.so,
myntulpe_C_ACLiC_dict_rdict.pcm, runjobs.C"
5 transfer_output_files = output
6 WhenToTransferOutput = ON_EXIT
7 Output = logfile/condor_test_$(Cluster)_$(Process).stdout
8 Error = logfile/condor_test_$(Cluster)_$(Process).stderr
9 Log = logfile/condor_test_$(Cluster)_$(Process).log
10 Queue 1

```

You can then use `condor_submit runOnCondor` to submit the file. Use `condor_q` when querying the status, where `held` is the abnormal status; if you want to remove the condor task, use `condor_rm` followed by `-all` to remove all tasks. Follow the ID of the corresponding task to remove the corresponding task. `condor_q -better-analyze 680724.207` and the ID of the corresponding task can check the reason for the condor hold. You can also use `condor_hold -all` to manually hold certain tasks, and `condor_release -all` to restart these held jobs.

```

zhufeng@node00:~/condor-ce-test$condor_q

-- Schedd: node00.hepnnu.com : <192.168.40.2:9618?... @ 09/01/25 13:35:21
OWNER  BATCH_NAME  SUBMITTED  DONE  RUN  IDLE  TOTAL JOB_IDS
zhufeng ID: 675979  9/1 13:35  _    2    _    2 675979.0-1

Total for query: 2 jobs; 0 completed, 0 removed, 0 idle, 2 running, 0 held, 0 suspended
Total for zhufeng: 2 jobs; 0 completed, 0 removed, 0 idle, 2 running, 0 held, 0 suspended
Total for all users: 4 jobs; 0 completed, 0 removed, 0 idle, 2 running, 2 held, 0 suspended

zhufeng@node00:~/condor-ce-test$condor_rm -all
All jobs have been marked for removal
zhufeng@node00:~/condor-ce-test$condor_submit sleep-condor.jdl
Submitting job(s)..
2 job(s) submitted to cluster 675980.
zhufeng@node00:~/condor-ce-test$condor_q

-- Schedd: node00.hepnnu.com : <192.168.40.2:9618?... @ 09/01/25 13:35:42
OWNER  BATCH_NAME  SUBMITTED  DONE  RUN  IDLE  TOTAL JOB_IDS
zhufeng ID: 675980  9/1 13:35  _    2    _    2 675980.0-1

Total for query: 2 jobs; 0 completed, 0 removed, 0 idle, 2 running, 0 held, 0 suspended
Total for zhufeng: 2 jobs; 0 completed, 0 removed, 0 idle, 2 running, 0 held, 0 suspended
Total for all users: 4 jobs; 0 completed, 0 removed, 0 idle, 2 running, 2 held, 0 suspended

zhufeng@node00:~/condor-ce-test$condor_rm 675980
All jobs in cluster 675980 have been marked for removal

```

In fact, everyone has their own way of submitting condor. Compared with CRAB, it is more free. You can start by learning to use other people's scripts, and then add some small ideas of your own. Proficient use of these two tools can greatly improve your research efficiency!

• Naming and automation of result output

As mentioned before, when submitting condor, you need to pay attention to the problem of overwriting the same name of the output file. So how to avoid this problem? Next, I will introduce how to make the output file have a different name.

In fact, the method is very simple. We only need to pass a variable of type `TString` (such as `outputname`) layer by layer in the code. You need to modify the following three places: so that you can use a script to generate multiple Runjobs.C to run at the same time and output them as different files. Finally, use `hadd` to combine these files into one.

The script can refer to `/home/storage0/users/zhufeng/formymultilep_learning/shell/`

```

1  //C
2  void myntuple::Loop(TString outputname){
3  .....
4  TString myroot = outputname + "_zhuf.root";
5  TFile* myhbk = new TFile (myroot,"recreate");
6
7  //h
8  virtual void      Loop(TString outputname);
9
10 //Runjobs
11 a.Loop("youroutputdir/youroutputname");

```

Before understanding this step, we must first understand the relationship between myntuple.C myntuple.h and Runjobs.C. When you use `MakeClass` to create a new myntuple.C, most of the member functions have been created, such as `GetEntry`. The functions of these functions are relatively simple, so they will be placed in the .h file. One of the `virtual void Loop();` and `Loop` member functions is the main function we need to write. If we get the myntuple_C.so file after compilation, it can be loaded and used in Runjobs.C (the path in the code below is a bit long and part of it has been omitted)

```

1  void Runjobs()
2  {
3      gSystem->Load("myntuple_C.so");//Load library
4      TChain * chain = new TChain("/mkcands/X_data","");//Create TChain for processing multiple TTree objects
5      chain->Add("../MuOnia/ReReco2016/FILE/mymultilep_*Number.root");//Load the root file in the path into TChain.
6      myntuple a(chain);//Use chain to create an instance of myntuple class a
7      a.Loop();//Call the Loop method of the myntuple class to process the data in TChain
8  }

```

It can be found that we will only call the `Loop` function in Runjobs. Therefore, we only need to add a `TString` variable in the brackets of the `Loop()` function. This variable can be passed to the `myntuple.C` internally and directly applied to the naming of the output file.

After mastering this, and using the Shell script to automatically replace `"youroutputname"` in `Runjobs.C`, your Condor tasks can be truly automated and run in parallel, and you no longer have to worry about file overwriting!

• Condor script submission

Since everyone's style is different, I will provide a general idea here. After all, today's AI tools are powerful enough, and only the older generation still insists on coding by hand.

• Split input file

First of all, there should be many root files waiting to be processed. If you add all these files into one `TChain`, it may take several days and nights for the Condor node to run alone. Moreover, once one of the files in the middle is broken or the node hangs, the entire chain will have to be rerun. So the first step is to think about how to split these root files to be processed and generate multiple different Runjobs.C files.

Usually these root files already have a good classification. In physical analysis, our original ROOT files usually exist in different subfolders according to the data set (Dataset), the running year (Era) or the running batch (Run/Job ID). So the first good way is to classify them according to the folder. You can use `find -maxdepth 1 -type d` (or pass specific wildcards) to find all these subfolders, so that a Condor Job only processes one folder. Here is an example:

```

1  #!/bin/bash
2  # Basic path and the data storage directory we want to scan
3  BASE_DIR="/home/users/zhufeng/"
4  DATA_DIR="/home/users/zhufeng/data"
5
6  # 1. Find all subfolders under DATA_DIR (the depth is 1, excluding the current directory itself, the depth
7  # should be adjusted according to the actual situation)
8  # The result of find will be: Run2016B_DoubleMuon, Run2016C_DoubleMuon, etc.
9  for dir_path in $(find "$DATA_DIR" -maxdepth 1 -mindepth 1 -type d); do
10
11      # Extract the name of the folder (such as Run2016B_DoubleMuon)

```

```

11     dir_name=$(basename "$dir_path")
12
13     # 2. Dynamically generate exclusive Runjobs.C
14     # We use sed to directly change the "path to be replaced" and "output file name" in the template.
15     cat <<EOF > "Runjobs_${dir_name}.C"
16 void Runjobs_${dir_name}()
17 {
18     gSystem->Load("myntuple_C.so");
19     TChain * chain = new TChain("/mkcands/X_data", "");
20
21     //Load all root files in the current subfolder
22     chain->Add("${dir_path}/*.root");
23
24     myntuple a(chain);
25
26     //Use the parameterized naming in the previous section and pass in the subtask name to distinguish the
    output file name.
27     a.Loop("output/${dir_name}");
28 }
29 EOF
30 done

```

Another way is to extract the paths of all root files `find "$DATA_DIR" -name "*.root" > all_files.txt`, and then split these root files using `split -l 10 all_files.txt split_`. This way, you can accurately control how many files each job runs based on the file size and computing power. After splitting, add the corresponding root file `Add` into the chain. My method is to create a basic file and then use `sed` in a loop to modify it:

```

1 cat <<EOL > Runjobs.C
2 void Runjobs()
3 {
4     gSystem->Load("myJJHINTupleZhuf_C.so");
5     TChain * chain = new TChain("mkcands/X_data", "");
6
7     //FILE
8
9     myJJHINTupleZhuf a(chain);
10    a.Loop("output/NAME");
11 }
12 EOL
13 for file in $(ls split); do
14     cp Runjobs.C runjobs/Runjobs_${file}.C
15     sed -i "s/Runjobs/Runjobs_${file}/g" runjobs/Runjobs_${file}.C
16     sed -i "s/NAME/${file}/g" runjobs/Runjobs_${file}.C
17     line=$(cat split/${file} | wc -l)
18     for ((n=1;n<=line;n++)); do
19         path=$(sed -n "${n}p" split/${file})
20         sed -i "0,/FILE/s|//FILE|chain->Add(\"${path}\");\n//FILE|" runjobs/Runjobs_${file}.C
21     done
22 done
23

```

• Run script & description file

After splitting Runjobs.C, the next very critical step is to write a running script. This script needs to initialize the CMS software environment, run ROOT for physical analysis, and may also need to transfer the output file to a specific path. Here is my script and condor description file for reference:

```

1 cat <<EOL > Run.sh
2 #!/bin/bash
3 source /cvmfs/cms.cern.ch/cmsset_default.sh
4 export SCRAM_ARCH=e19_amd64_gcc12
5 export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/lib64:/srv
6 eval `scramv1 project CMSSW CMSSW_14_0_4`

```

```

7  cd CMSSW_14_0_4/src/
8  eval `scramv1 runtime -sh`
9  cd ../../
10 mkdir -p output
11 root -b -q -l `ls`
12 cp output/* ${outputpath}
13 # rm -f output/*
14 EOL
15
16 cat <<EOL > runOnCondor
17 universe = vanilla
18 # Executable = runjobs/SCRIPT
19 Executable = runjobs/Run.sh
20 Arguments = `ls`
21 Should_Transfer_Files = YES
22 Transfer_Input_Files = ${transfer_files[@]}
23 WhenToTransferOutput = ON_EXIT
24 Output = logfile/condor_test_`ls`_${Process}.stdout
25 Error = logfile/condor_test_`ls`_${Process}.stderr
26 Log = logfile/condor_test_`ls`_${Process}.log
27 # Queue 1
28
29 Queue filename matching runjobs/Runjobs_*.C
30 # This line of command will scan all files in the runjobs/ directory that match the Runjobs_*.C format
31 # For each matched file, Condor will automatically generate a Job for it (`ls` increases from 0)
32 # And assign the current file name to the `ls` variable and pass it to the above Arguments and
   Transfer_Input_Files
33 EOL

```

Through `Queue filename matching runjobs/Runjobs_*.C`, Condor will allocate an independently running `Process` for each split `Runjobs_*.C`. When transferring files, each `Process` will only get its own path list of 10 ROOT files and run code, without redundancy.

8-two muons & two tracks

Before starting Chapter 9, we can do a small project to help with subsequent understanding. This time we will rebuild the process of $\psi(2S) \rightarrow J/\psi + \pi^+ \pi^-$, $J/\psi \rightarrow \mu^+ \mu^-$. Compared with the previous two In the reconstruction of muons, two newly charged track pions are added. If we want them to decay from the same vertex, that is, the mother particle $\psi(2S)$, we need to add two more tracks for vertex fitting on the basis of the previous one. The following is a comparison of the two parts. Next, we only need to perform a similar loop on the track~~ It seems~~ we can find the mother particle we want!

```

vector < RefCountedKinematicParticle > muonParticles;
muonParticles.push_back(pmumuFactory.particle(muonPTT,
muon_mass, chi, ndf, muon_sigma));
muonParticles.push_back(pmumuFactory.particle(muonMTT,
muon_mass, chi, ndf, muon_sigma));
KinematicParticleVertexFitter fitter;
RefCountedKinematicTree psiVertexFitTree;
psiVertexFitTree = fitter.fit(muonParticles);
if ( !(psiVertexFitTree->isValid()) ) {continue;}
psiVertexFitTree->movePointerToTheTop();
RefCountedKinematicParticle psi_vFit_noMC =
psiVertexFitTree->currentParticle();
RefCountedKinematicVertex psi_vFit_vertex_noMC =
psiVertexFitTree->currentDecayVertex();

```

```

vector < RefCountedKinematicParticle > JPiPiParticles;
JPiPiParticles.push_back(JPiPiFactory.particle(trackTT1,
pion_mass, chi, ndf, pion_sigma));
JPiPiParticles.push_back(JPiPiFactory.particle(trackTT2,
pion_mass, chi, ndf, pion_sigma));
JPiPiParticles.push_back(pmumuFactory.particle(muon1TT,
muon_mass, chi, ndf, muon_sigma));
JPiPiParticles.push_back(pmumuFactory.particle(muon2TT,
muon_mass, chi, ndf, muon_sigma));
KinematicParticleVertexFitter fitter;
RefCountedKinematicTree JPiPiVertexFitTree;
JPiPiVertexFitTree = fitter.fit(JPiPiParticles);
if ( !(JPiPiVertexFitTree->isValid()) ) {continue;}
JPiPiVertexFitTree->movePointerToTheTop();
RefCountedKinematicParticle JPiPi_vFit_noMC =
JPiPiVertexFitTree->currentParticle();
RefCountedKinematicVertex JPiPi_vFit_vertex_noMC =
JPiPiVertexFitTree->currentDecayVertex();

```

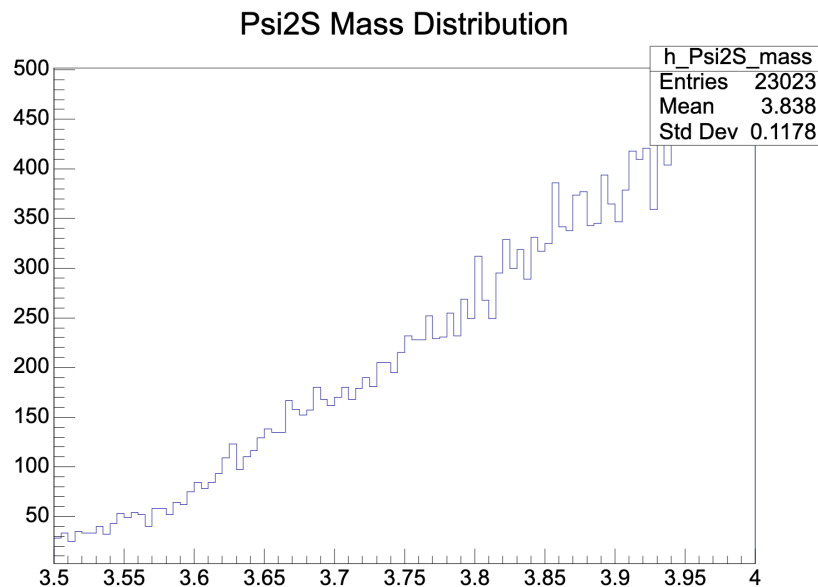
For track tokens, you need to obtain them in this way. Please consider where they should be placed. Refer to muon tokens.

```

1  ///in .h file
2  edm::EDGetTokenT<edm::View<pat::PackedCandidate>> trackToken_;
3  //edm::EDGetTokenT<edm::View<reco::Track>> trackToken_;
4  ///
5
6
7  ///in .cc file
8  trackToken_ = consumes<edm::View<pat::PackedCandidate>>(edm::InputTag("packedPFCandidates"));
9  //trackToken_ = consumes<edm::View<reco::Track>>(edm::InputTag("displacedTracks"));
10 ///.....///
11 edm::Handle<edm::View<pat::PackedCandidate>> theTrackHandle;
12 // edm::Handle<edm::View<reco::Track>> theTrackHandle;
13 iEvent.getByToken(trackToken_,theTrackHandle );
14
15 ///.....///
16 // for (edm::View<reco::Track>::const_iterator iTrack1 = theTrackHandle->begin(); iTrack1 != theTrackHandle-
17 >end(); ++iTrack1) {
18   for (edm::View<pat::PackedCandidate>::const_iterator iTrack1 = theTrackHandle->begin(); iTrack1 !=
19   theTrackHandle->end(); ++iTrack1) {
20     if (((iTrack1->px()==iMuon2->track()->px() && iTrack1->py()==iMuon2->track()->py() &&
21     iTrack1->pz()==iMuon2->track()->pz()) || ( iTrack1->px()==iMuon1->track()->px() && iTrack1->py()==iMuon1-
22     >track()->py() && iTrack1->pz()==iMuon1->track()->pz())) {continue;} //Prevent this charged track from being
23     left by muon

```

When you output `JPiPi_vFit_noMC->currentState().mass()` like two muons, you can get the constant mass spectrum of $J/\psi\pi^+\pi^-$. The mass of $\psi(2S)$ is 3.686GeV, then you can get the constant mass spectrum of $J/\psi\pi^+$ A peak is seen at the 3.6GeV position of the constant mass spectrum of π^- (?)



When you use the above code to process multiple datasets and finally get a result with sufficient statistics, you find that the constant mass spectrum does not have the $\psi(2S)$ peak at the 3.6GeV position as expected, similar to the picture above. Is it a code problem or is the condition not tight enough? One main reason is that what we want to find is $\psi(2S) \rightarrow J/\psi + \pi^+\pi^-$, $J/\psi \rightarrow \mu^+\mu^-$. In the process of μ^- , just two μ and two π can fit to a vertex does not mean that two μ are decayed from J/ψ , which results in a large part of them being just background. To avoid this situation, there are two conceivable methods. One is that the two μ I selected have constant masses near the mass of J/ψ . Based on the mass spectrum of $\mu\mu$ only done before, I can roughly choose one of $3.0\text{GeV} < \text{Two } \mu \text{ in the mass range of } M_{\mu^+\mu^-} < 3.2\text{GeV}$, and let them perform vertex fitting with two π , seems to be able to reduce the background events a lot. But I always feel that there are some strange things in it... When you go to [PDG](#) to query the information of J/ψ particles, you will find that its decay width is only 0.0000926GeV! On our constant mass spectrum, the width of the

peak at 3.096GeV is obviously much larger than this value. WHY? Is there something wrong with our detector?

In fact, this is affected by the resolution of the detector. Logically speaking, in the real physical world, if we can accurately measure the momentum of μ , the shape of the J/ψ peak we reconstruct should be like a δ function. But our detector cannot do this, and the momentum it can measure has errors, and because this process is: μ hits the detector $\rightarrow$ leaves the spatial position of the electrical signal $\rightarrow$ fits multiple electrical signals to form a trajectory $\rightarrow$ calculates the momentum of μ based on the radius of the trajectory. The detectors are like pixels. If they light up, it means that the pixel has been here. Obviously, this pixel cannot be infinitely fine. This is called resolution, just like the difference between 4K and 360P. Since there is an inaccuracy caused by the resolution, it will also cause the J/ψ mass to become "blurred" (when you take off your glasses and look at the moon, you will see a lump of moon), so how to correct this? We need to do mass constraint! Please refer to [this section](#) for the impact of detector resolution.

As the name suggests, we need to re-constrain the fuzzy $\mu\mu$ momentum near J/ψ to 3.096GeV through some **fitting corrections**! One thing to note here is that not all particles are suitable for mass constraint. For example, particles with a large decay width are not suitable, which will change the original physical properties. After understanding this, let's start writing code!

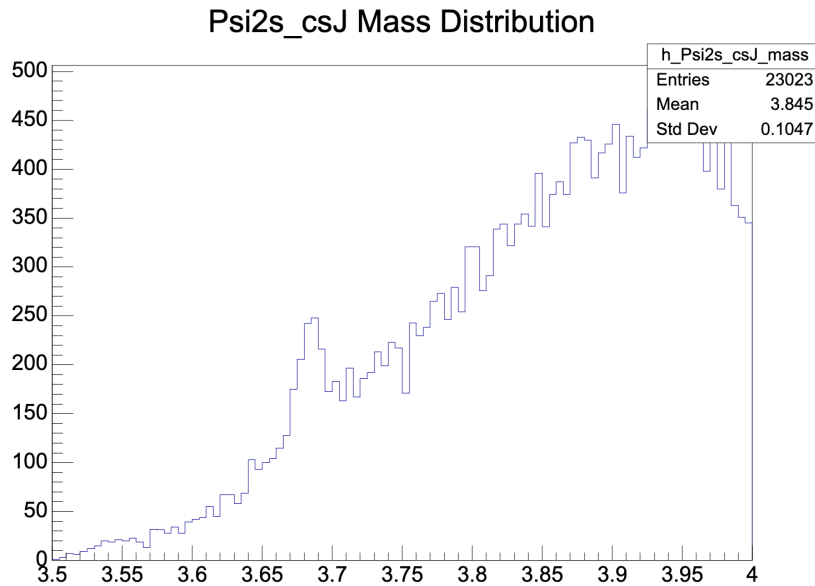
```

1  TransientTrack muon1TT(muTrack1, &(bFieldHandle));
2  TransientTrack muon2TT(muTrack2, &(bFieldHandle));
3  KinematicParticleFactoryFromTransientTrack pmumuFactory;
4  ParticleMass muon_mass = myMumass;    // pdg mass
5  float muon_sigma = myMumasserr;
6  float chi = 0.;
7  float ndf = 0.;
8  vector < RefCountedKinematicParticle > muonParticles;
9  muonParticles.push_back(pmumuFactory.particle(muon1TT, muon_mass, chi, ndf, muon_sigma));
10 muonParticles.push_back(pmumuFactory.particle(muon2TT, muon_mass, chi, ndf, muon_sigma));
11 KinematicParticleVertexFitter fitter;
12 RefCountedKinematicTree jpsilVertexFitTree;
13 jpsilVertexFitTree = fitter.fit(muonParticles);
14
15 if (!jpsilVertexFitTree->isValid()) {continue;}
16 jpsilVertexFitTree->movePointerToTheTop();
17 RefCountedKinematicParticle jpsil_vFit_noMC = jpsilVertexFitTree->currentParticle();
18 RefCountedKinematicVertex jpsil_vFit_vertex_noMC = jpsilVertexFitTree->currentDecayVertex();
19 KinematicParameters mymumupara= jpsil_vFit_noMC->currentState().kinematicParameters();
20 double jpsil_vtxprob=ChiSquaredProbability( (double)(jpsil_vFit_vertex_noMC->chiSquared()),(double)
(jpsil_vFit_vertex_noMC->degreesOfFreedom()));
21 //if(jpsil_vFit_noMC->currentState().mass()>3.4 || jpsil_vFit_noMC->currentState().mass()<2.8) {continue;}
22 //if( jpsil_vtxprob<vtxprobbprecut) {continue;}
23 //The above is consistent with mumuonly
24
25 //mass constrain for jpsil from psi2s:
26 const double myJmass=3.0969, myJmasserr=0.00004;
27 KinematicConstraint *jpsi_cs = new MassKinematicConstraint(myJmass,myJmasserr);//Set the particle parameters
you want to constrain
28 KinematicParticleFitter csfitter;
29 jpsilVertexFitTree = csfitter.fit(jpsi_cs,jpsilVertexFitTree);
30 jpsilVertexFitTree->movePointerToTheTop();
31 RefCountedKinematicParticle jpsil_vFit_cs = jpsilVertexFitTree->currentParticle();
32 RefCountedKinematicVertex jpsil_vFit_vertex_cs = jpsilVertexFitTree->currentDecayVertex();
33 KinematicParameters mymumupara_cs= jpsil_vFit_cs->currentState().kinematicParameters();
34
35
36 //Fit the J/psi particles that have passed the mass constraint to the two tracks //
37 vector < RefCountedKinematicParticle > csJ_PiPiParticles;
38 csJ_PiPiParticles.push_back(JPiPiFactory.particle(trackTT1, pion_mass, chi, ndf, pion_sigma));
39 csJ_PiPiParticles.push_back(JPiPiFactory.particle(trackTT2, pion_mass, chi, ndf, pion_sigma));
40 csJ_PiPiParticles.push_back(jpsil_vFit_cs);
41
42 KinematicParticleVertexFitter fitter_csJ;
43 RefCountedKinematicTree JPiPiVertexFitTree_csJ;
44 JPiPiVertexFitTree_csJ = fitter_csJ.fit(csJ_PiPiParticles);
45 if( !(JPiPiVertexFitTree_csJ->isValid()) ) {continue;}
46 JPiPiVertexFitTree_csJ->movePointerToTheTop();
47 RefCountedKinematicParticle JPiPi_vFit_csJ = JPiPiVertexFitTree_csJ->currentParticle();

```

```
48 RefCountedKinematicVertex JPiPi_vFit_vertex_csJ = JPiPiVertexFitTree_csJ->currentDecayVertex();
```

In this way, after we output the constant mass spectrum of $J/\psi \pi\pi$ after mass constraint, we can clearly see a sharp peak near 3.686GeV. Of course, further research is needed to reduce the background signal. If you are interested, you may be able to find another small peak at 3.872GeV, which is the famous X(3872). But that's another story, and our story continues...



9-four muons fitting

p.s. If you copy the following content directly, some variables may not be mentioned or defined, because I will only introduce the logic, and some details may not be fully included due to my carelessness. However, I believe that you can completely deal with these minor problems after the previous exercises!

In the previous analysis, we successfully fit two muons and obtained some exciting results. But our journey will not stop here. In the next chapters we will study four muons. Doubling the number of muons brings more variables and more complex screening, as well as some more interesting results!

```
1  if (thePATMuonHandle->size()>=4){
2      for (iMuon1 = thePATMuonHandle->begin(); iMuon1 != thePATMuonHandle->end(); ++iMuon1) {
3          TrackRef muTrack1 = iMuon1->track();
4          if (muTrack1.isNull()) {continue;}
5          reco::Track recoMu1 = *iMuon1->track();
6          for (iMuon2 = iMuon1 + 1; iMuon2 != thePATMuonHandle->end(); ++iMuon2) {
7              TrackRef muTrack2 = iMuon2->track();
8              if (muTrack2.isNull()) {continue;}
9              reco::Track recoMu2 = *iMuon2->track();
10             for (iMuon3 = iMuon2 + 1; iMuon3 != thePATMuonHandle->end(); ++iMuon3) {
11                 TrackRef muTrack3 = iMuon3->track();
12                 if (muTrack3.isNull()) {continue;}
13                 reco::Track recoMu3 = *iMuon3->track();
14                 for (iMuon4 = iMuon3 + 1; iMuon4 != thePATMuonHandle->end(); ++iMuon4) {
15                     TrackRef muTrack4 = iMuon4->track();
16                     if (muTrack4.isNull()) {continue;}
17                     reco::Track recoMu4 = *iMuon4->track();
18                     if ( ( iMuon1->charge()+iMuon2->charge()+iMuon3->charge()+iMuon4->charge() ) == 0 ) {
```

The loop is similar to the previous one, performing subsequent pairing operations by traversing all μ in an instance. You can add a condition `if (thePATMuonHandle->size()>=4)` before this to ensure that there are indeed four or more μ in your case, so that you can perform subsequent operations. Similarly, if we want to reconstruct two neutral particles, we need to require that the charge sum of the four μ is zero, so the μ that can pass this condition must be $\mu^+ \mu^- \mu^+ \mu^-$, but in the program we only get $\mu_1 \mu_2 \mu_3 \mu_4$, different from two μ , four μ means that if two are combined, we will have three different combinations

$$\mu_1 \mu_2, \mu_3 \mu_4 \mid \mu_1 \mu_3, \mu_2 \mu_4 \mid \mu_1 \mu_4, \mu_2 \mu_3$$

Among these three combinations, consider that the sum of the charges of the two μ in the pairwise combination should also be 0, so in the end only two combinations are correct. Please note that the charge ordering of $\mu_1\mu_2\mu_3\mu_4$ is not necessarily $\mu^+\mu^-\mu^+\mu^-$, so it does not mean that $\mu_1\mu_3, \mu_2\mu_4$ must be the wrong combination (you need to think!) Then in order to eliminate the wrong combinations, you can choose to filter in mytuple.C, or you can filter in .cc. **** The former is our final main processing method. For the sake of logical continuity, I will show the processing method in .cc, and also pave the way for subsequent more advanced processing methods. ****

In fact, it is very simple, just fill in a two-dimensional array with the sum of charges equal to zero.

```

1  if ( (iMuon1->charge()+iMuon2->charge()+iMuon3->charge()+iMuon4->charge() ) == 0 ) {
2      std::vector<std::pair<const edm::View<pat::Muon>::const_iterator, const
edm::View<pat::Muon>::const_iterator>> muonPairs;
3      if ((iMuon1->charge() + iMuon2->charge()) == 0) {
4          muonPairs.push_back(std::make_pair(iMuon1, iMuon2));
5          muonPairs.push_back(std::make_pair(iMuon3, iMuon4));
6      }
7      if ((iMuon1->charge() + iMuon3->charge()) == 0) {
8          muonPairs.push_back(std::make_pair(iMuon1, iMuon3));
9          muonPairs.push_back(std::make_pair(iMuon2, iMuon4));
10     }
11     if ((iMuon1->charge() + iMuon4->charge()) == 0) {
12         muonPairs.push_back(std::make_pair(iMuon1, iMuon4));
13         muonPairs.push_back(std::make_pair(iMuon2, iMuon3));
14     }
15     for (unsigned int i = 0; i < muonPairs.size(); i += 2) {
16         const auto& muon1 = muonPairs[i].first;
17         const auto& muon2 = muonPairs[i].second;
18         const auto& muon3 = muonPairs[i+1].first;
19         const auto& muon4 = muonPairs[i+1].second;
20         //The following code is very similar to the two muon parts. I believe you can easily understand the
meaning.
21         TransientTrack muonPTT1(muon1->track(), &(bFieldHandle));
22         TransientTrack muonMTT2(muon2->track(), &(bFieldHandle));
23         TransientTrack muonPTT3(muon3->track(), &(bFieldHandle));
24         TransientTrack muonMTT4(muon4->track(), &(bFieldHandle));
25         KinematicParticleFactoryFromTransientTrack pmumuFactory;
26         ParticleMass muon_mass = myMumass;
27         float muon_sigma = myMumasserr;
28         float chi = 0.;
29         float ndf = 0.;
30         vector < RefCountedKinematicParticle > muonParticles1;
31         muonParticles1.push_back(pmumuFactory.particle(muonPTT1, muon_mass, chi, ndf, muon_sigma));
32         muonParticles1.push_back(pmumuFactory.particle(muonMTT2, muon_mass, chi, ndf, muon_sigma));
33         muonParticles1.push_back(pmumuFactory.particle(muonPTT3, muon_mass, chi, ndf, muon_sigma));
34         muonParticles1.push_back(pmumuFactory.particle(muonMTT4, muon_mass, chi, ndf, muon_sigma));
35         KinematicParticleVertexFitter fitter;
36         RefCountedKinematicTree psiVertexFitTree1;
37         psiVertexFitTree1 = fitter.fit(muonParticles1);
38
39         if (psiVertexFitTree1->isValid()) {
40             //Here I deleted the part that outputs fourmuonmass and directly outputs the momentum of each muon.
41             //The order of child is the same as the order of filling in muonParticles1, so it is consistent with the
order of muon1
42             psiVertexFitTree1->movePointerToTheFirstChild();
43             RefCountedKinematicParticle mu1CandMC = psiVertexFitTree1->currentParticle();
44             KinematicParameters myFourMuonMu1KP= mu1CandMC->currentState().kinematicParameters();
45             psiVertexFitTree1->movePointerToTheNextChild();
46             RefCountedKinematicParticle mu2CandMC = psiVertexFitTree1->currentParticle();
47             KinematicParameters myFourMuonMu2KP= mu2CandMC->currentState().kinematicParameters();
48             psiVertexFitTree1->movePointerToTheNextChild();
49             RefCountedKinematicParticle mu3CandMC = psiVertexFitTree1->currentParticle();
50             KinematicParameters myFourMuonMu3KP= mu3CandMC->currentState().kinematicParameters();
51             psiVertexFitTree1->movePointerToTheNextChild();
52             RefCountedKinematicParticle mu4CandMC = psiVertexFitTree1->currentParticle();
53             KinematicParameters myFourMuonMu4KP= mu4CandMC->currentState().kinematicParameters();

```

```

54
55 //The following needs to output the momentum of the four muon, so that we can obtain the four momentum of
these muon
56 MyFourMuonMu1Px->push_back(myFourMuonMu1KP.momentum().x());
57 MyFourMuonMu1Py->push_back(myFourMuonMu1KP.momentum().y());
58 MyFourMuonMu1Pz->push_back(myFourMuonMu1KP.momentum().z());
59 MyFourMuonMu2Px->push_back(myFourMuonMu2KP.momentum().x());
60 MyFourMuonMu2Py->push_back(myFourMuonMu2KP.momentum().y());
61 MyFourMuonMu2Pz->push_back(myFourMuonMu2KP.momentum().z());
62 MyFourMuonMu3Px->push_back(myFourMuonMu3KP.momentum().x());
63 MyFourMuonMu3Py->push_back(myFourMuonMu3KP.momentum().y());
64 MyFourMuonMu3Pz->push_back(myFourMuonMu3KP.momentum().z());
65 MyFourMuonMu4Px->push_back(myFourMuonMu4KP.momentum().x());
66 MyFourMuonMu4Py->push_back(myFourMuonMu4KP.momentum().y());
67 MyFourMuonMu4Pz->push_back(myFourMuonMu4KP.momentum().z());
68
69 mumuonlymu1Idx->push_back(std::distance(thePATMuonHandle->begin(), muon1));
70 mumuonlymu2Idx->push_back(std::distance(thePATMuonHandle->begin(), muon2));
71 mumuonlymu3Idx->push_back(std::distance(thePATMuonHandle->begin(), muon3));
72 mumuonlymu4Idx->push_back(std::distance(thePATMuonHandle->begin(), muon4));

```

If you don't want to perform this filtering step in .cc, just replace the corresponding `muon1` with `iMuon1`. But you also need to output the charge corresponding to μ for later screening in myntuple.C, in the [Muon ID and Muon block](#) chapter, the charges of all μ have been output in order, so you can use the `(*muCharge)[(*mumuonlymu1Idx)[myi]]` method to obtain the charges corresponding to μ in myntuple.C (this is just as a reminder, please think about which variable `size()` is taken for myi here)

In this way, processing in myntuple.C becomes easier. Please use the following tips to obtain the mass of your $\mu\mu$ combination, so that you can draw two constant mass spectra, or even draw a two-dimensional distribution diagram!

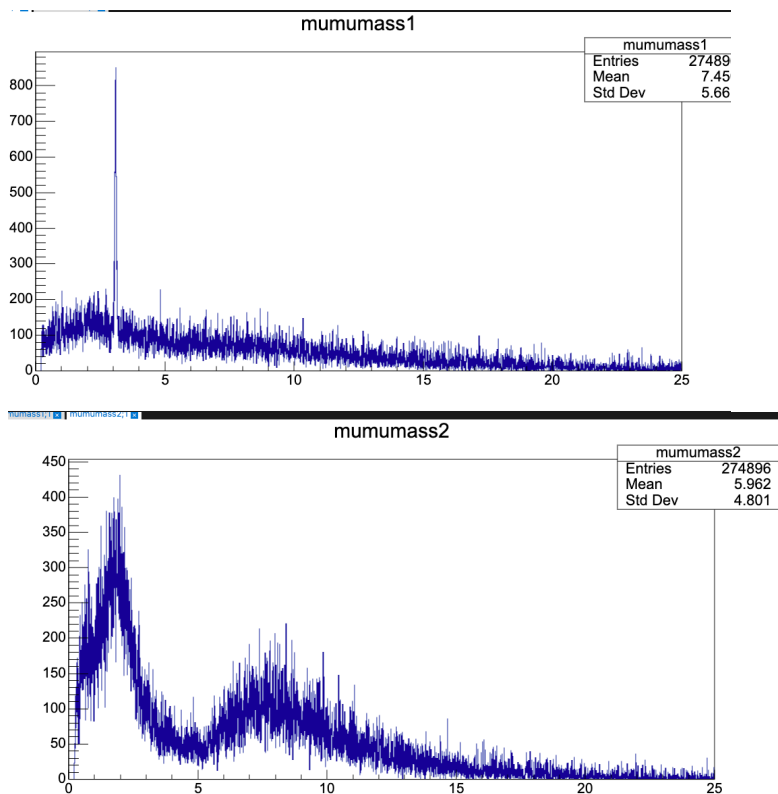
```

1 #include "TLorentzVector.h"
2 TH2F* mumumassJJ = new TH2F("mumumassJJ", "mumumassJJ", 60, 2.8, 3.4, 60, 2.8, 3.4);
3 ...
4 TLorentzVector Muon1FourVector;
5 Muon1FourVector.SetXYZM((*muPx)[(*MyFourMuonMu1Idx)[myi]], (*muPy)[(*MyFourMuonMu1Idx)[myi]], (*muPz)
[(*MyFourMuonMu1Idx)[myi]], muonMass);
6 std::cout<<Muon1FourVector.M() << Muon1FourVector.Pt() << std::endl;
7 ...
8 mumumassJJ->Fill(mumumass1, mumumass2);

```

However, *だがしかし*, you may find that the constant mass of your second pair μ looks strange, why can the peak be clearly seen in the first pair, but not in the second pair? The following is my preliminary explanation. You can use some methods to verify whether what I said is correct.

The momentum of the μ elements in a case is sorted from large to small. The two μ elements in the first pair tend to get larger transverse momentum pt and thus have a lower background. Therefore, if you want to see the peak in the second pair μ , you can limit the pt of μ

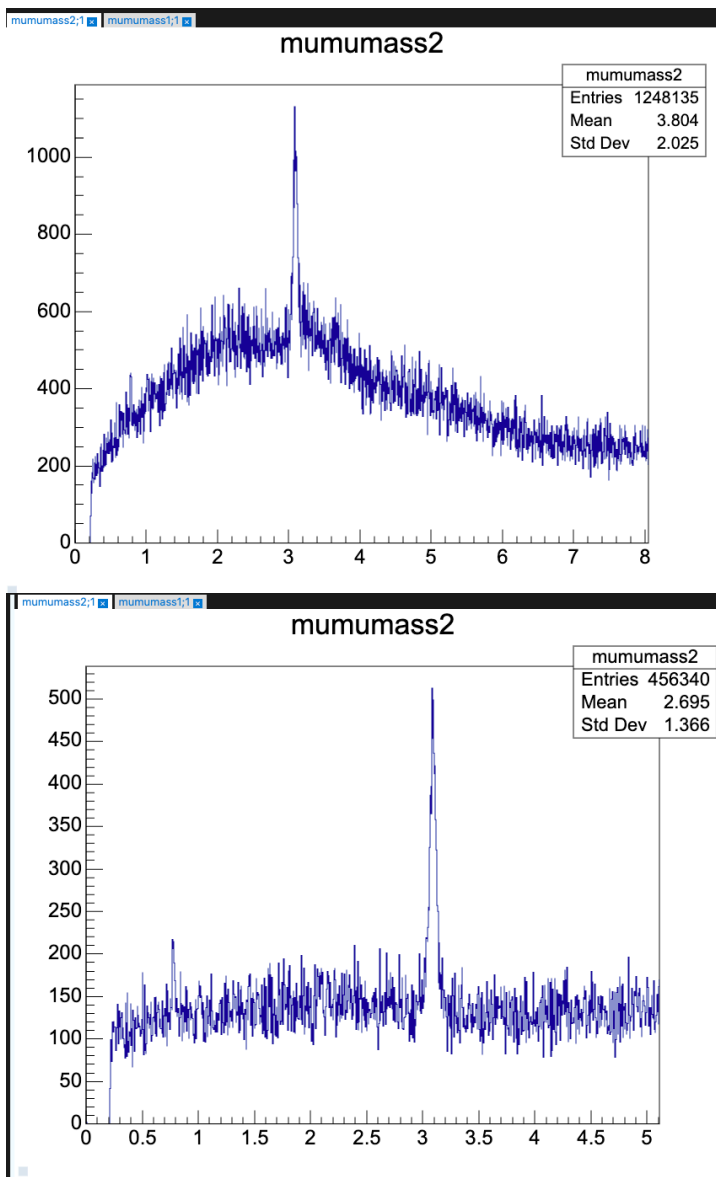


However, when the background is reduced, it also means that there will be a corresponding loss of the signal. Therefore, the data of one root file is not enough. You need to try to run several more files. In [cms Data Aggregation System](#), I have filtered out the required root files. You can use a command similar to the following to download it to your directory.

```
xrdcp -d 1 -f root://xrootd-cms.infn.it//store/data/Run2023B/ParkingDoubleMuonLowMass0/MINIAOD/22Sep2023-v1/2560000/0cb0075f-5308-4849-a0ee-f451012a0c7c.root .
```

After downloading the appropriate number of files (about 10), modify the inputFiles in runMultiLepPAT_dataRun3_miniAOD.py. After getting the data, you can set the pt of each μ sub in myntuple.C to be greater than 1.5 or 2 and you should be able to see the peak! In the picture below, the left side is `pt>1.5` and the right side is `pt>2`

```
1 ivars.inputFiles=(
2 'file:f8e93985-e14c-4a8a-b28b-f8cceb3c878e.root', #You can parallelize multiple root files by adding commas
3 'file:847912fd-20d4-4cbe-80bf-3eeade7bcb1f.root',
4 'root://xrootd-cms.infn.it//store/data/Run2023B/ParkingDoubleMuonLowMass0/MINIAOD/22Sep2023-v1/2560000/0cb0075f-5308-4849-a0ee-f451012a0c7c.root'#You can also directly write the path on das without
  downloading
5 )
```



10-mass constraint

Note: Before that, the relevant files are included in the folder below. .cc is all the code up to the 4muon part. You can overwrite it with the .cc you originally wrote yourself. .h contains all the required variables. It doesn't matter if these variables are not used. There is also a complete code as a reminder. There are many repeated structures in this part. After understanding the logic, please confirm the variable names carefully and be patient!

Use the file in the path below!

/afs/cern.ch/user/z/zhuf/public/UserCode/massconst

Or /nnucms/home/zhufeng/formutilep_learning/UserCode/massconst

```
--->MuMuEEPat.cc
--->MuMuEEPat.h
--->MultiLepPAT.cc_hint
```

Some loop structures have been adjusted in the new .cc file to make the overall logic more concise and clear.

```
-- for iMuonP Here, traverse all  $\mu$  and output all information
-> raw_muonPx, muonID, Index...

if Handle->size()>=2: If there are more than 2  $\mu$ 
-- for iMuon1 iMuon2
-> mumuonly fit_muonPx
if Handle->size()>=4: If there are more than 4  $\mu$ 
```

```
-- for iMuon3 iMuon4
-> fourmuon
-> mass constraint for each pairs (we are here!)
```

Earlier in [four-muon fitting](#) we mentioned that there are three different pairing combinations:

$\mu_1\mu_2, \mu_3\mu_4 \mid \mu_1\mu_3, \mu_2\mu_4 \mid \mu_1\mu_4, \mu_2\mu_3$

Then if we need to do mass constraint on J/ψ reconstructed from each pair of μ , we need to repeat it three times according to different charge conditions. For example, we do the first group first:

```
1  if(muon1TT.charge()+muon2TT.charge()==0) {
2      muonP12.push_back(pFactory.particle(muon1TT, muon_mass, chi, ndf, muon_sigma));
3      muonP12.push_back(pFactory.particle(muon2TT, muon_mass, chi, ndf, muon_sigma));
4      muonP34.push_back(pFactory.particle(muon3TT, muon_mass, chi, ndf, muon_sigma));
5      muonP34.push_back(pFactory.particle(muon4TT, muon_mass, chi, ndf, muon_sigma));
6      RefCountedKinematicTree Jpsi1 = kpvFitter.fit(muonP12);
7      RefCountedKinematicTree Jpsi2 = kpvFitter.fit(muonP34);
8
9      RefCountedKinematicTree Jpsi1noMCJJ = kpvFitter.fit(muonP12);
10     RefCountedKinematicTree Jpsi2noMCJJ = kpvFitter.fit(muonP34);
11     if (Jpsi1->isValid() && Jpsi2->isValid()){
12         .....//This part is to take the physical information of two J/\psi respectively. It is similar to
the previous one and will not be explained too much.
13         if(doJpsiMassCost) { //This part is mass constraint. You can remove this if without judging.
14             RefCountedKinematicTree Chi1_bTree;
15             RefCountedKinematicParticle MyChi1_part;
16
17             //JJ assumption
18             jp1 = myJmass; jp_m_sigm1 = myJmasserr;
19             jp2 = myJmass; jp_m_sigma2 = myJmasserr;
20             jpsi_c1 = new MassKinematicConstraint(jp1,jp_m_sigm1);
21             jpsi_c2 = new MassKinematicConstraint(jp2,jp_m_sigma2); //Set the particle parameters you want to
constrain
22             try{ Jpsi1 = csFitter.fit(jpsi_c1,Jpsi1noMCJJ); } catch (VertexException const& x) { cout<<"mu12 vertex
exception with mass constrained to J!"<<endl; }
23             try{ Jpsi2 = csFitter.fit(jpsi_c2,Jpsi2noMCJJ); } catch (VertexException const& x) { cout<<"mu34 vertex
exception with mass constrained to J!"<<endl; } //Try to constrain the previous mumu combination to the
parameters you set, and overwrite the result into Jpsi1
24             if(Jpsi1->isEmpty()!=true && Jpsi2->isEmpty()!=true) { //If Jpsi1 and Jpsi2 are successfully fitted and
non-empty, extract the current particles and add them to the Chi_1 list
25                 Jpsi1->movePointerToTheTop();
26                 Jpsi2->movePointerToTheTop();
27                 Jpsi1_part = Jpsi1->currentParticle();
28                 Jpsi2_part = Jpsi2->currentParticle();
29                 Chi_1.push_back(Jpsi1_part);
30                 Chi_1.push_back(Jpsi2_part);
31                 bool isagoodfit=true;
32                 try{ Chi1_bTree = kpvFitter.fit(Chi_1); } catch (VertexException const& x) {isagoodfit=false;
cout<<"mu12 and mu34 vertex exception with mu12 and mu34 constrained to JJ"<<endl; } //It is equivalent to
first fitting the vertices of the two muons to J/\psi, and then fitting the two J/\psi after mass constraint.
33                 if(Chi1_bTree->isValid() && isagoodfit) { //If the fitting is successful, extract J/\psi physical
information
34                     .....//Omit
35                 }
36             }
37             delete jpsi_c1; delete jpsi_c2;
38             //end JJ assumption
```

Note: About the definition of Ctau

A Lorentz invariant is needed to represent the lifetime of the particle.

Under the laboratory department: $\tau = \gamma\tau_0$; τ_0 is the static life of the particle $\gamma = \frac{1}{\sqrt{1-\beta^2}}, \beta = \frac{v}{c}$

Decay length: $L = v\tau = \gamma\beta c\tau_0$; where $c\tau_0$ can be considered Lorentz invariant

Among them $p = \gamma m_0 \beta$, bring in to get $c\tau_0 = \frac{L}{\gamma\beta} = \frac{Lm_0}{p}$

It should be noted that $c\tau_0$ here is when the direction of momentum and the direction of displacement are parallel. When there is a certain angle, Lorentz contraction will only occur when it is parallel to the direction of displacement. Therefore, in actual calculations, there is also an included angle $\cos\alpha$. The following is the function definition that can be found in .h.

```

1  virtual double GetcTau(RefCountedKinematicVertex& decayVrtx, RefCountedKinematicParticle& kinePart, Vertex&
   bs)
2  { TVector3 vtx;
3    TVector3 pvtx;
4    vtx.SetXYZ((*decayVrtx).position().x(), (*decayVrtx).position().y(), 0);
5    pvtx.SetXYZ(bs.position().x(), bs.position().y(), 0);
6    VertexDistanceXY vdistXY;
7    TVector3 pperp(kinePart->currentState().globalMomentum().x(),
8                  kinePart->currentState().globalMomentum().y(), 0);
9
10   TVector3 vdiff = vtx - pvtx;
11   double cosAlpha = vdiff.Dot(pperp) / (vdiff.Perp() * pperp.Perp());
12   Measurement1D distXY = vdistXY.distance(Vertex(*decayVrtx), Vertex(bs));
13   double ctauPV = distXY.value() * cosAlpha * kinePart->currentState().mass() / pperp.Perp();
14   return ctauPV;
15 }
```

The above is basically the end of the content of .cc. When we look at the full version of the code, we will find that in addition to JJ's mass constraint, there are many other decay channel codes, and there are not many differences between those codes, so we will not repeat them here. However, learning does not end here. The filtering conditions in .cc are only the most basic ones. Its main function is to do some fitting and output the corresponding physical quantities. The more detailed filtering is completed in myntuple.C. Then in the next chapters I will continue to introduce the content of myntuple.C.

11-myntuple advanced

Note: The following tasks are recommended to be completed in the Tsinghua cluster. You can use the mymultilep.root file in the path below to perform the `MakeClass` operation to generate and write your myntuple.C file.

Data path: Here is the data file you can use. You need to find the variable name you need in the previous full version of cc.
 /home/storage0/users/llchen/dataMINI/Run3/ReReco2023/2023B/ParkingDoubleMuonLowMass0
 myntuple_hint: This is the integrated file of all decay channels. It is very messy. Select the part you need.
 /home/storage0/users/zhufeng/formymultilep_learning/myntuple.C_hint

Management and sorting of data input

In the previous myntuple.C, we learned some simple drawing operations, which obviously cannot meet our growing screening needs. With the addition of four muon and mass constraint, more variables have been brought, and we also need to manage these variables. Here is an idea:

In .cc, the variables of μ are output in groups of four. Likewise, in .C, we will obtain them with the same idea. For example, the charge and fourLorentzVector of a μ child can be input into an array in groups of four: (The following is not code, just as a logical display)

We can fill the μ physical quantities into the array in order:
 MuCharge: {charge_mu1, charge_mu2, charge_mu3, charge_mu4}
 Mu4vect: {4vect_mu1, 4vect_mu2, 4vect_mu3, 4vect_mu4}

If all the variables are filled into a corresponding array in this order, then we only need **variable name + serial number** to get the physical quantity corresponding to μ . For example, if we want to know the Px of the third μ , we only need to use `(MuPx)[2]` to get it. The premise is that you create the `MuPx` array and fill in the variables in the correct order.

Now that we have managed the physical quantities of the four μ sub-subs in an appropriate way, the next step is to pair the four μ sub-subs. Previously, I introduced the method of pairing them by using the `if` condition to determine whether the charge sum is zero. If it is zero, the constant masses of the two μ after pairing will be output. It is also known that only two of the three pairing methods are suitable. This method is indeed very convenient to get what we want under few conditions, but we need to find a common label to filter and combine these physical quantities. The above management is to prepare for this step.

Do you still remember the serial numbers just mentioned? In fact, if we combine and sort the serial numbers, then after completing the operation, we only need to find the corresponding physical quantities and output them! for example:

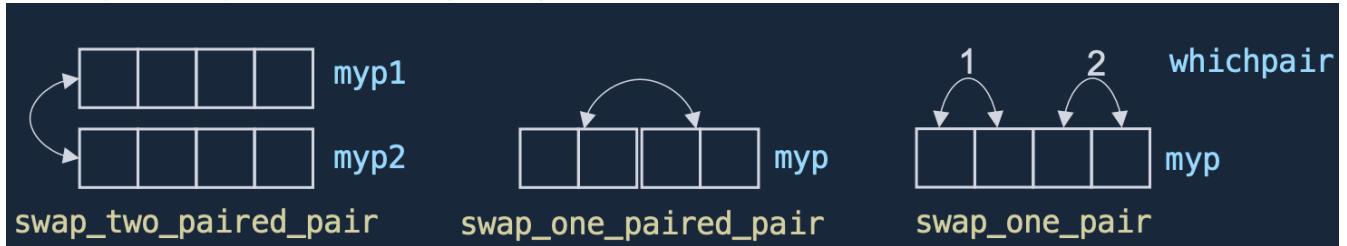
Combine the serial numbers: `myCombIdx[3] = {0,1,2,3}, {0,2,1,3}, {0,3,1,2},`
 Corresponding to $\mu_1\mu_2, \mu_3\mu_4 \mid \mu_1\mu_3, \mu_2\mu_4 \mid \mu_1\mu_4, \mu_2\mu_3$ The combination of μ_3

For example, we want to see whether the combination $\mu_1\mu_3$ satisfies the condition that the charge sum is zero. If it meets, the constant mass of $\mu_1\mu_3$ will be output:

```
1 if ( (MuCharge)[(myCombIdx)[1][0]] + (MuCharge)[(myCombIdx)[1][1]] == 0)
2 { ( (Mu4vect)[(myCombIdx)[1][0]] + (Mu4vect)[(myCombIdx)[1][1]] ).M() }
```

Next is the sorting. In addition to solving the specification problem, there are also decay channels such as $\Upsilon J/\psi$. We need to make the constant mass of the two μ particles of the first pair greater than that of the second pair. This is more reasonable. Below I will introduce the ordering of charge and mass:

First find the wrong charge combination and put it into `myCombIdx[0]` with the sequence number being zero, so that if we don't want to study the charged state, we can just loop over `[1][2]`, then compare the constant masses of the two pairs, put the larger one in front, that is, `(([0]+[1]).M() > ([2]+[3]).M())`, and then arrange the charges of μ in the order of `+-+-`. To achieve the above requirements, you need the following exchange functions.



In addition to these basic sorting, we can also sort based on how close their mass is to the mass of the particle you are looking for. The χ^2 test will be used here.

Chi-square value χ^2 (Chi-Square)

The chi-square value is a statistic that measures the deviation between an observed value and an expected value. The larger the chi-square value, the greater the deviation between the observed value and the expected value.

For example, if I want to study the decay path $\psi(2S)J/\psi$, I can use the formula of χ^2 :

$$\chi^2 = \left(\frac{M_{1_{obs}} - M_{\psi(2S)}}{\sigma_{M_1}} \right)^2 + \left(\frac{M_{2_{obs}} - M_{J/\psi}}{\sigma_{M_2}} \right)^2$$

In the specific code, in order to adjust the impact of the quality error, a scaling factor is multiplied after the quality error.

In addition to the functions used for sorting, there is also a function `setFourMuPairsVars` that corresponds to the invariant quality of the $\mu\mu$ pair. The function of this function is to correspond the sorted `myCombIdx` to the invariant quality of the $\mu\mu$ pair that has been overfitted in cc. For example, the `myCombIdx[1]` after sorting is `{3,1,4,2}`, then the `myFourMuVars[1][0] = (*MyJpsi3Var_Mu13)` `[fourMuIdx] myFourMuVars[1][1] = (*MyJpsi4Var_Mu24)[fourMuIdx]` in the array of `myFourMuVars[3][2]` I output

Sort mule pairs

Although the method of exchanging serial numbers in the original code is easy to understand, the code is not easy to read. Here is a new sorting idea.

First of all, our goal is, after pairing two of the four $\mu^{+/-}$ with a total charge of 0 - 1. Select the combination we want with a charge of 0, 2. Be able to sort the physical quantities of a single μ in each combination, 3. Be able to sort according to the physical quantities of the two combinations.

In fact, the pairings have been sorted during our preprocessing process. We can think of it as the default order. This default order must have a one-to-one correspondence for other variables, so we need to ensure that it can still correspond to all variables after we perform some sorting. One consideration is to sort all variables at the same time during sorting, which is the way of the original code. One way to consider the sorting sequence numbers is to read them in a specific order when reading variables.

For the default order, we can use a 6*2 matrix to mark it, and add extra columns for each row to explain, and for the corresponding variables, it is a 6*n matrix. In order to facilitate the correspondence with the program, I use the serial number of the array instead of the actual serial number in the matrix, that is, 0 represents the first one.

0	1					<i>parameter</i>	<i>charge</i>	<i>sort</i>	<i>Xparameter</i>
2	3	0	1	0	1	P_{12}	T	T	X_{1234}
---	---	2	3	2	3	P_{23}	T	F	X_{1234}
0	2	→ 0	2	→ 0	2	P_{13}	F	F	X_{1324}
1	3	1	3	1	3	P_{24}	F	T	X_{1324}
---	---	0	3	0	3	P_{14}	T	T	X_{1423}
0	3	1	2	1	2	P_{23}	T	F	X_{1423}
1	2								
---	---								

For example, if we want to determine that the charge of two μ elements is zero, we only need to create a 6*1 bool matrix simultaneously, so that we can know whether the corresponding combination has a charge of 0. For the sorting of μ itself, only two serial numbers need to be exchanged. Such an exchange will not cause misalignment, because the physical information of the μ subchild is obtained in an independent list through its own serial number. As for sorting the physical quantities of two μ subcombinations, you only need to create a 6*1 bool matrix. For example, if you want the larger p_T of the two combinations to be placed first, then after comparing their p_T and getting the bool amount, in the subsequent loop, you can use the corresponding bool value to ensure that the physical quantity corresponding to the combination with the larger p_T is output first. For physical quantities that require 4 μ , such as the mass of

```

1  int comb[6][2] = {
2      {0,1},{2,3},{0,2},{1,3},{0,3},{1,2}
3  };
4  bool jpsiValid[6] = {false};
5  bool sortjpsi[6] = {false};
6  for (int i = 0; i < 6; i+=2) {
7      int m1 = comb[i][0];
8      int m2 = comb[i][1];
9      int m3 = comb[i+1][0];
10     int m4 = comb[i+1][1];
11     if (fitMuCharge[m1] + fitMuCharge[m2] == 0) jpsiValid[i] = true;
12     if (fitMuCharge[m3] + fitMuCharge[m4] == 0) jpsiValid[i+1] = true;
13
14     // first muon with higher pt
15     if (rawMup4vect[m1].Pt() < rawMup4vect[m2].Pt()) {
16         comb[i][0] = m2; comb[i][1] = m1;
17     }
18     if (rawMup4vect[m3].Pt() < rawMup4vect[m4].Pt()) {
19         comb[i+1][0] = m4; comb[i+1][1] = m3;
20     }
21
22     // first jpsi with higher pt/?
23     if ((rawMup4vect[m1]+rawMup4vect[m2]).Pt() > (rawMup4vect[m3]+rawMup4vect[m4]).Pt()) {
24         sortjpsi[i] = true;
25         sortjpsi[i+1] = false;
26     } else {
27         sortjpsi[i+1] = true;
28         sortjpsi[i] = false;

```

```

29     }
30 }
31
32 double JpsiMass[6]=  {(*MyJpsi1Mass_Mu12)[myFourMuIdx],  (*MyJpsi2Mass_Mu34)[myFourMuIdx],
(*MyJpsi3Mass_Mu13)[myFourMuIdx],  (*MyJpsi4Mass_Mu24)[myFourMuIdx],  (*MyJpsi5Mass_Mu14)[myFourMuIdx],
(*MyJpsi6Mass_Mu23)[myFourMuIdx]  };
33
34 for (int i = 0; i < 6; i+=2) {
35     if (jpsiValid[i]){
36         int leadIdx = sortjpsi[i] ? i : i + 1;
37         int subIdx  = sortjpsi[i] ? i + 1 : i;
38         myYJ2SMC << std::fixed
39             << JpsiMass[leadIdx] << " " << JpsiMass[subIdx] << " " << std::endl;
40     }
41 }

```

Tricks

- how to check luminosity from json file

```

1 $ source /cvmfs/cms-bril.cern.ch/brilconda310/bin/activate
2 $ python -m pip install --user brilws
3 Collecting brilws
4   Downloading brilws-3.9.4.tar.gz (54 kB)
5     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 54.8/54.8 kB 723.7 kB/s eta 0:00:00
6   Preparing metadata (setup.py) ... done
7 Building wheels for collected packages: brilws
8   Building wheel for brilws (setup.py) ... done
9   Created wheel for brilws: filename=brilws-3.9.4-py3-none-any.whl size=66416
sha256=640fae6400de3386bbadb967ed7bcf5875a767253ecab9916900688fd3ebd9d
10   Stored in directory:
   /uscms/homes/f/fengzhu/.cache/pip/wheels/d0/70/a0/6c55074594134a17d0bd1683ac59b2fa9027b75d838569c373
11 Successfully built brilws
12 Installing collected packages: brilws
13   WARNING: The scripts brilcalc, brilschema and briltag are installed in '/uscms/home/fengzhu/.local/bin'
   which is not on PATH.
14   Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-
   location.
15 Successfully installed brilws-3.9.4
16 $ export PATH=$HOME/.local/bin:$PATH
17 $ which brilcalc
18 ~/.local/bin/brilcalc
19 $ brilcalc --version
20 3.9.4
21 $ brilcalc lumi -c web -i Collisions26HI_5p36TeV_404337_404926_muon.json -u /nb

```

end-